

PROYEK AKHIR SEMESTER GASAL



NAMA : Noval Akbar
KELAS : X PPLG 1
NIS : 258725

JUDUL PROYEK : APLIKASI CUACA

PENGEMBANGAN PERANGKAT LUNAK DAN GIM
SMK NEGERI 1 KANDEMAN
TAHUN PELAJARAN 2025/2026

DESKRIPSI PROYEK

Proyek ini bertujuan untuk membuat **Aplikasi Cuaca** berbasis desktop menggunakan C# Windows Forms yang dapat menampilkan informasi cuaca real-time dari berbagai lokasi di seluruh dunia. Aplikasi ini mengintegrasikan **WeatherAPI** untuk mengambil data cuaca terkini dan ramalan cuaca beberapa hari ke depan.

Fitur Utama Aplikasi:

1. Pencarian Lokasi

- Pengguna dapat memasukkan nama kota untuk mencari informasi cuaca
- Mendukung pencarian kota di seluruh dunia

2. Informasi Cuaca Saat Ini

- Menampilkan suhu dalam Celsius
- Kondisi cuaca (cerah, berawan, hujan, dll)
- Suhu yang dirasakan (feels like)
- Icon cuaca yang dinamis sesuai kondisi dan waktu (siang/malam)

3. Data Detail Cuaca

- **Indeks UV:** Tingkat radiasi ultraviolet
- **Kelembaban:** Persentase kelembaban udara
- **Tekanan Udara:** Dalam millibar (mb)
- **Tutupan Awan:** Persentase awan
- **Kecepatan Angin:** Dalam km/h beserta arah angin
- **Curah Hujan:** Dalam millimeter (mm)

4. Informasi Kualitas Udara

- Indeks kualitas udara (Air Quality Index)
- PM 2.5 (partikel halus)
- PM 10 (partikel kasar)
- Carbon Monoxide (CO)
- Ozone (O3)

5. Ramalan Cuaca 3 Hari

- Tanggal ramalan
- Suhu maksimal dan minimal
- Kondisi cuaca
- Icon cuaca untuk setiap hari

6. Icon Dinamis

- Icon berubah otomatis berdasarkan kondisi cuaca

- Perbedaan icon untuk siang dan malam hari
- Icon indikator untuk berbagai parameter cuaca

Konsep Pemrograman yang Diimplementasikan:

1. **Async/Await Programming** - Untuk operasi asynchronous saat mengambil data dari API
2. **HTTP Client** - Untuk melakukan request ke WeatherAPI
3. **JSON Deserialization** - Parsing data JSON dari API menggunakan Newtonsoft.Json
4. **Class dan Object** - Membuat class model untuk data cuaca (Root, Current, Location, dll)
5. **Dictionary** - Mapping kode cuaca ke nama icon
6. **GUI (Graphical User Interface)** - Antarmuka visual menggunakan Windows Forms
7. **Error Handling** - Try-catch untuk menangani error
8. **Event Handling** - Menangani event button click dan key press
9. **Static Class dan Method** - Class WeatherIcon sebagai utility class
10. **LINQ dan Collections** - Menggunakan List dan Dictionary untuk menyimpan data

RINGKASAN TEORI C# YANG DIGUNAKAN

1. Async/Await Programming

Async/await digunakan untuk operasi yang membutuhkan waktu (seperti request ke API) agar aplikasi tidak freeze.

```
private async Task GetWeatherData(string city)
{
    HttpResponseMessage response = await httpClient.GetAsync(url);
    string json = await response.Content.ReadAsStringAsync();
}
```

Penjelasan:

- async - Menandai method sebagai asynchronous
- await - Menunggu operasi selesai tanpa memblokir UI
- Task - Representasi operasi asynchronous

2. HTTP Client

HttpClient digunakan untuk berkomunikasi dengan web API.

```
private HttpClient httpClient;
httpClient = new HttpClient();
HttpResponseMessage response = await httpClient.GetAsync(url);
```

Komponen:

- HttpClient - Class untuk mengirim HTTP request
- GetAsync() - Method untuk HTTP GET request
- HttpResponseMessage - Response dari server

3. JSON Deserialization

Mengubah data JSON dari API menjadi object C#.

```
string json = await response.Content.ReadAsStringAsync();  
Root weatherData = JsonConvert.DeserializeObject<Root>(json);
```

Library: Newtonsoft.Json (Json.NET)

4. Class dan Object

Membuat struktur data untuk merepresentasikan data cuaca.

```
public class Current  
{  
    public required string last_updated { get; set; }  
    public required object temp_c { get; set; }  
    public required int is_day { get; set; }  
    public required Condition condition { get; set; }  
    public required double wind_kph { get; set; }  
    public required string wind_dir { get; set; }  
    public required double pressure_mb { get; set; }  
    public required double precip_mm { get; set; }  
    public required double humidity { get; set; }  
    public required double cloud { get; set; }  
    public required double feelslike_c { get; set; }  
    public required double uv { get; set; }  
    public required double gust_mph { get; set; }  
    public required object gust_kph { get; set; }  
    public required AirQuality air_quality { get; set; }  
}
```

Konsep:

- required - Property wajib diisi
- { get; set; } - Auto-implemented property

5. Dictionary

Menyimpan mapping antara kode cuaca dengan nama icon.

```
private static Dictionary<int, string> weatherCodeToIcon = new Dictionary<int,  
string>();  
{  
    // Cerah/Berawan  
    { 1000, "clear-day" },  
  
    // Cerah Berawan  
    { 1003, "partly-cloudy-day" },  
  
    // Berawan  
    { 1006, "cloudy" },  
    { 1009, "overcast" },  
  
    // Kabut  
    { 1030, "mist" },  
    { 1135, "fog" },  
    { 1147, "fog" },  
}
```



```

// Gerimis
{ 1063, "drizzle" },
{ 1150, "drizzle" },
{ 1153, "drizzle" },
{ 1168, "sleet" },
{ 1171, "sleet" },

// Hujan
{ 1180, "rain" },
{ 1183, "rain" },
{ 1186, "rain" },
{ 1189, "rain" },
{ 1192, "rain" },
{ 1195, "rain" },
{ 1198, "sleet" },
{ 1201, "sleet" },
{ 1240, "rain" },
{ 1243, "rain" },
{ 1246, "rain" },

// Salju
{ 1066, "snow" },
{ 1069, "sleet" },
{ 1072, "sleet" },
{ 1114, "snow" },
{ 1117, "snow" },
{ 1204, "sleet" },
{ 1207, "sleet" },
{ 1210, "snow" },
{ 1213, "snow" },
{ 1216, "snow" },
{ 1219, "snow" },
{ 1222, "snow" },
{ 1225, "snow" },
{ 1237, "hail" },
{ 1249, "sleet" },
{ 1252, "sleet" },
{ 1255, "snow" },
{ 1258, "snow" },
{ 1261, "hail" },
{ 1264, "hail" },

// Badai Petir
{ 1087, "thunderstorms-day" },
{ 1273, "thunderstorms-day-rain" },
{ 1276, "thunderstorms-rain" },
{ 1279, "thunderstorms-day-snow" },
{ 1282, "thunderstorms-snow" }
};

```

6. Switch Expression

Menentukan deskripsi kualitas udara berdasarkan indeks.

```

private string GetAQIDescription(int index)
{
    return index switch
    {
        1 => "Good", // Baik
        2 => "Moderate", // Sedang
        3 => "Unhealthy for Sensitive Groups", // Tidak sehat untuk kelompok
sensitif

```

```

        4 => "Unhealthy", // Tidak sehat
        5 => "Very Unhealthy", // Sangat tidak sehat
        6 => "Hazardous", // Berbahaya
        _ => "Not Available" // Tidak tersedia
    };
}

```

7. Event Handling

Menangani event dari UI seperti button click.

```

private async void btnSearch_Click(object sender, EventArgs e)
{
    string city = txtCity.Text.Trim();
    await GetWeatherData(city);
}

```

8. Conditional Statements

Menentukan icon yang tepat berdasarkan kondisi.

```

if (data.current.pressure_mb >= 1013.25)
{
    picPressure.LoadAsync(Path.Combine(Application.StartupPath, "icons",
    "pressure-high.png"));
}
else
{
    picPressure.LoadAsync(Path.Combine(Application.StartupPath, "icons",
    "pressure-low.png"));
}

// Load icon angin (alert jika kecepatan >= 21 km/h)
if (data.current.wind_kph >= 21)
{
    picWind.LoadAsync(Path.Combine(Application.StartupPath, "icons", "wind-
    alert.png"));
}
else
{
    picWind.LoadAsync(Path.Combine(Application.StartupPath, "icons",
    "wind.png"));
}

// Load icon curah hujan (banyak tetes jika >= 11 mm)
if (data.current.precip_mm >= 11)
{
    picPrecip.LoadAsync(Path.Combine(Application.StartupPath, "icons",
    "raindrops.png"));
}
else
{
    picPrecip.LoadAsync(Path.Combine(Application.StartupPath, "icons",
    "raindrop.png"));
}

```

```

// Load icon UV berdasarkan indeks UV (0-11+)
int uvIndex = (int)Math.Floor(data.current.uv);
uvIndex = Math.Min(uvIndex, 11); // Cap maksimal di 11

string iconName = uvIndex >= 1
    ? $"uv-index-{uvIndex}.png"
    : "not-available.png";

picUV.LoadAsync(Path.Combine(Application.StartupPath, "icons", iconName));

// Load icon awan berdasarkan persentase awan
if (data.current.cloud <= 50)
{
    picCloud.LoadAsync(Path.Combine(Application.StartupPath, "icons",
    "mist.png"));
}
else
{
    picCloud.LoadAsync(Path.Combine(Application.StartupPath, "icons",
    "overcast.png"));
}

```

9. String Interpolation

Membuat string dinamis dengan mudah.

```

string url =
    $"{FORECAST_API_URL}?key={API_KEY}&q={city}&days=5&aqi=yes&alerts=yes";

```

10. Try-Catch Exception Handling

Menangani error yang mungkin terjadi.

```

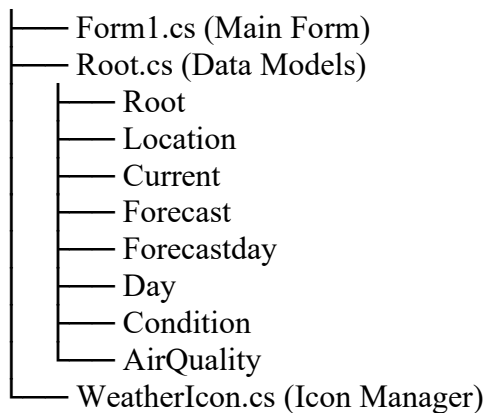
try
{
    HttpResponseMessage response = await httpClient.GetAsync(url);
    // Kirim request
}
catch (Exception ex)
{
    MessageBox.Show($"Error: {ex.Message}", "Error",
        MessageBoxButton.OK, MessageBoxIcon.Error);
}

```

PERANCANGAN PROGRAM

Struktur Class

WeatherApp



Alur Kerja Program

1. **User Input** → User memasukkan nama kota di TextBox
2. **Validation** → Program mengecek apakah input kosong
3. **API Request** → Program mengirim request ke WeatherAPI
4. **JSON Response** → API mengembalikan data dalam format JSON
5. **Deserialization** → JSON diubah menjadi object C#
6. **Display Data** → Data ditampilkan ke UI (Label, PictureBox)
7. **Icon Loading** → Icon cuaca dimuat dari folder local

API Endpoint

http://api.weatherapi.com/v1/forecast.json?key={API_KEY}&q={city}&days=5&aqi=yes&alerts=yes

Parameter:

- key - API Key untuk autentikasi
- q - Query lokasi (nama kota)
- days - Jumlah hari forecast (5 hari)
- aqi - Include air quality data (yes)
- alerts - Include weather alerts (yes)

IMPLEMENTASI PROGRAM

1. Form1.cs - Main Application

```
using Newtonsoft.Json;
using System;
using System.Drawing;
using System.Globalization;
using System.IO;
using System.Net.Http;
using System.Threading.Tasks;
using System.Windows.Forms;
using WeatherApp;

namespace WeatherApp
{
    public partial class Form1 : Form
    {
        // Konstanta API
        private const string API_KEY = "xxxxxxxxxxxxxxxxxx";
        private const string FORECAST_API_URL =
"http://api.weatherapi.com/v1/forecast.json";
        private HttpClient httpClient;

        // PictureBox untuk icon
        private PictureBox pictureBoxWeather;
        private PictureBox picDay1, picDay2, picDay3;

        public Form1()
        {
            InitializeComponent();
            httpClient = new HttpClient();
            string iconFolder = Path.Combine(Application.StartupPath, "icons");
            WeatherIcon.IconFolder(iconFolder);
            InitializeWeatherIcons();
        }

        // Inisialisasi PictureBox
        private void InitializeWeatherIcons()
        {
            pictureBoxWeather = new PictureBox();
            pictureBoxWeather.Location = new Point(35, 47);
            pictureBoxWeather.Size = new Size(140, 140);
            pictureBoxWeather.SizeMode = PictureBoxSizeMode.Zoom;
            pictureBoxWeather.BackColor = Color.Transparent;
            panelForecast.Controls.Add(pictureBoxWeather);
            pictureBoxWeather.BringToFront();

            picDay1 = CreateForecastIcon(panelDay1);
            picDay2 = CreateForecastIcon(panelDay2);
            picDay3 = CreateForecastIcon(panelDay3);
        }

        // Helper untuk membuat forecast icon
        private PictureBox CreateForecastIcon(Panel panel)
        {
            var pictureBox = new PictureBox();
            pictureBox.Location = new Point(53, 23);
            pictureBox.Size = new Size(80, 80);
            pictureBox.SizeMode = PictureBoxSizeMode.Zoom;
            pictureBox.BackColor = Color.Transparent;
            panel.Controls.Add(pictureBox);
            panel.BringToFront();
        }
    }
}
```

```

        return pictureBox;
    }

    // Event handler tombol Search
    private async void btnSearch_Click(object sender, EventArgs e)
    {
        string city = txtCity.Text.Trim();

        if (string.IsNullOrEmpty(city))
        {
            MessageBox.Show("Please enter a city name!", "Warning",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }

        btnSearch.Enabled = false;
        btnSearch.Text = "Loading...";
        lblStatus.Text = "Fetching weather data...";
        lblStatus.ForeColor = Color.White;

        await GetWeatherData(city);

        btnSearch.Enabled = true;
        btnSearch.Text = "Search";
    }

    // Fungsi untuk mengambil data dari API
    private async Task GetWeatherData(string city)
    {
        try
        {
            string url =
                $"{FORECAST_API_URL}?key={API_KEY}&q={city}&days=5&aqi=yes&alerts=yes";
            HttpResponseMessage response = await httpClient.GetAsync(url);

            if (response.IsSuccessStatusCode)
            {
                string json = await response.Content.ReadAsStringAsync();
                Root weatherData =
                    JsonConvert.DeserializeObject<Root>(json)!;
                DisplayWeatherData(weatherData);
            }
            else
            {
                lblStatus.Text = "City not found!";
                lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
                ClearWeatherDisplay();
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Error: {ex.Message}", "Error",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
            lblStatus.Text = "Error loading data";
            lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
        }
    }

    // Fungsi untuk menampilkan data ke UI
    private void DisplayWeatherData(Root data)
    {
        if (data == null) return;
    }

```

```

// Data lokasi
lblCity.Text = data.location.name;
lblCountry.Text = $"{data.location.region}, {data.location.country}";
lblLastUpdate.Text = $"Last updated: \n{data.current.last_updated}";

// Data cuaca saat ini
lblTemp.Text = $"{data.current.temp_c}°C";
lblCondition.Text = data.current.condition.text;
lblFeelsLike.Text = $"Feels like: {data.current.feelslike_c}°C";

// Data detail
lblUVValue.Text = data.current.uv.ToString();
lblHumidityValue.Text = $"{data.current.humidity}%";
lblPressureValue.Text = $"{data.current.pressure_mb} mb";
lblCloudValue.Text = $"{data.current.cloud}%";
lblWindValue.Text = $"{data.current.wind_kph}
km/h\n{data.current.wind_dir}";
lblPrecipValue.Text = $"{data.current.precip_mm} mm";

// Data kualitas udara
if (data.current.air_quality != null)
{
    lblAQValue.Text = $"Air Quality
\n{GetAQIDescription(data.current.air_quality.usepaindex)}";
    lblPM25Value.Text = $"PM 2.5: {data.current.air_quality.pm2_5:F1}
µg/m³";
    lblPM10Value.Text = $"PM10: {data.current.air_quality.pm10:F1}
µg/m³";
    lblCOValue.Text = $"CO: {data.current.air_quality.co:F1} µg/m³";
    lblO3Value.Text = $"O3: {data.current.air_quality.o3:F1} µg/m³";
}

// Load icon cuaca
LocalIcon(data.current.condition.code, data.current.is_day == 1,
pictureBoxWeather);

// Tampilkan forecast 3 hari
if (data.forecast.forecastday != null &&
data.forecast.forecastday.Count >= 3)
{
    DisplayForecastDay(data.forecast.forecastday[0], lblDay1Date,
lblDay1Temp, lblDay1Condition, picDay1);
    DisplayForecastDay(data.forecast.forecastday[1], lblDay2Date,
lblDay2Temp, lblDay2Condition, picDay2);
    DisplayForecastDay(data.forecast.forecastday[2], lblDay3Date,
lblDay3Temp, lblDay3Condition, picDay3);
}

// Load icon detail cuaca
LoadDetailIcons(data);

lblStatus.Text = "Weather data loaded successfully!";
lblStatus.ForeColor = Color.FromArgb(150, 255, 150);
}

// Fungsi untuk menampilkan forecast per hari
private void DisplayForecastDay(Forecastday forecastDay, Label lblDate,
Label lblTemp, Label lblCondition, PictureBox pictureBox)
{
    if (forecastDay == null) return;

    DateTime date = DateTime.Parse(forecastDay.date);

```

```

        lblDate.Text = date.ToString("ddd, MMM dd",
CultureInfo.InvariantCulture);
        lblTemp.Text =
${forecastDay.day.maxtemp_c:F0}°/{forecastDay.day.mintemp_c:F0}°";
        lblCondition.Text = forecastDay.day.condition.text;

        LocalIcon(forecastDay.day.condition.code, true, pictureBox);
    }

    // Fungsi untuk load icon cuaca
    private void LocalIcon(int weatherCode, bool isDay, PictureBox
pictureBox)
    {
        try
        {
            Image icon = WeatherIcon.LoadWeatherIcon(weatherCode, isDay);
            if (icon != null)
                pictureBox.Image = icon;
            else
                pictureBox.Image = null;
        }
        catch
        {
            pictureBox.Image = null;
        }
    }

    // Fungsi untuk mendapatkan deskripsi AQI
    private string GetAQIDescription(int index)
    {
        return index switch
        {
            1 => "Good",
            2 => "Moderate",
            3 => "Unhealthy for Sensitive Groups",
            4 => "Unhealthy",
            5 => "Very Unhealthy",
            6 => "Hazardous",
            _ => "Not Available"
        };
    }

    // Fungsi untuk membersihkan tampilan
    private void ClearWeatherDisplay()
    {
        lblCity.Text = "-";
        lblCountry.Text = "-";
        lblTemp.Text = "-";
        lblCondition.Text = "-";
        lblFeelsLike.Text = "Feels like: -";
        // ... reset semua label lainnya
    }

    // Event handler ketika user tekan Enter
    private void txtCity_KeyPress(object sender, KeyPressEventArgs e)
    {
        if (e.KeyChar == (char)Keys.Enter)
        {
            btnSearch_Click(sender, e);
            e.Handled = true;
        }
    }

```



```

        // Cleanup saat form ditutup
        protected override void OnFormClosing(FormClosingEventArgs e)
        {
            httpClient?.Dispose();
            base.OnFormClosing(e);
        }
    }
}

```

2. Root.cs - Data Models

```

using Newtonsoft.Json;

namespace WeatherApp
{
    // Class untuk kualitas udara
    public class AirQuality
    {
        public required object co { get; set; }
        public required object no2 { get; set; }
        public required object o3 { get; set; }
        public required object so2 { get; set; }
        public required object pm2_5 { get; set; }
        public required object pm10 { get; set; }

        [JsonProperty("us-epa-index")]
        public required int usepaindex { get; set; }

        [JsonProperty("gb-defra-index")]
        public required int gbdefraindex { get; set; }
    }

    // Class untuk kondisi cuaca
    public class Condition
    {
        public required string text { get; set; }
        public required string icon { get; set; }
        public required int code { get; set; }
    }

    // Class untuk cuaca saat ini
    public class Current
    {
        public required object last_updated_epoch { get; set; }
        public required string last_updated { get; set; }
        public required object temp_c { get; set; }
        public required object temp_f { get; set; }
        public required int is_day { get; set; }
        public required Condition condition { get; set; }
        public required object wind_mph { get; set; }
        public required double wind_kph { get; set; }
        public required object wind_degree { get; set; }
        public required string wind_dir { get; set; }
        public required double pressure_mb { get; set; }
        public required double pressure_in { get; set; }
        public required double precip_mm { get; set; }
        public required double precip_in { get; set; }
        public required double humidity { get; set; }
        public required double cloud { get; set; }
        public required double feelslike_c { get; set; }
        public required double feelslike_f { get; set; }
        public required double vis_km { get; set; }
    }
}

```

```

        public required double vis_miles { get; set; }
        public required double uv { get; set; }
        public required double gust_mph { get; set; }
        public required object gust_kph { get; set; }
        public required AirQuality air_quality { get; set; }
    }

    // Class untuk lokasi
    public class Location
    {
        public required string name { get; set; }
        public required string region { get; set; }
        public required string country { get; set; }
        public required object lat { get; set; }
        public required object lon { get; set; }
        public required string tz_id { get; set; }
        public required object localtime_epoch { get; set; }
        public required string localtime { get; set; }
    }

    // Class untuk forecast
    public class Forecast
    {
        public required List<Forecastday> forecastday { get; set; }
    }

    // Class untuk forecast per hari
    public class Forecastday
    {
        public required string date { get; set; }
        public required int date_epoch { get; set; }
        public required Day day { get; set; }
    }

    // Class untuk detail cuaca harian
    public class Day
    {
        public required double maxtemp_c { get; set; }
        public required double maxtemp_f { get; set; }
        public required double mintemp_c { get; set; }
        public required double mintemp_f { get; set; }
        public required double avgtemp_c { get; set; }
        public required double avgtemp_f { get; set; }
        public required double maxwind_mph { get; set; }
        public required double maxwind_kph { get; set; }
        public required double totalprecip_mm { get; set; }
        public required double totalprecip_in { get; set; }
        public required double avgvis_km { get; set; }
        public required double avgvis_miles { get; set; }
        public required int avghumidity { get; set; }
        public required int daily_will_it_rain { get; set; }
        public required int daily_chance_of_rain { get; set; }
        public required int daily_will_it_snow { get; set; }
        public required int daily_chance_of_snow { get; set; }
        public required Condition condition { get; set; }
        public required double uv { get; set; }
    }

    // Class Root sebagai container utama
    public class Root
    {
        public required Location location { get; set; }
        public required Current current { get; set; }
    }

```

```

        public required Forecast forecast { get; set; }
        public required object alert { get; set; }
    }
}

```

3. WeatherIcon.cs - Icon Manager

```

using System;
using System.Collections.Generic;
using System.Drawing;
using System.IO;
using System.Windows.Forms;

namespace WeatherApp
{
    public static class WeatherIcon
    {
        private static Dictionary<int, string> weatherCodeToIcon = new
Dictionary<int, string>();
        private static string iconFolder = Path.Combine(Application.StartupPath,
"icons");

        static WeatherIcon()
        {
            InitializeWeatherCodes();
        }

        public static void IconFolder(string folderPath)
        {
            iconFolder = folderPath;
        }

        // Inisialisasi mapping kode cuaca ke icon
        private static void InitializeWeatherCodes()
        {
            weatherCodeToIcon = new Dictionary<int, string>
            {
                { 1000, "clear-day" },
                { 1003, "partly-cloudy-day" },
                { 1006, "cloudy" },
                { 1009, "overcast" },
                { 1030, "mist" },
                { 1135, "fog" },
                { 1147, "fog" },
                { 1063, "drizzle" },
                { 1150, "drizzle" },
                { 1153, "drizzle" },
                { 1180, "rain" },
                { 1183, "rain" },
                { 1186, "rain" },
                { 1189, "rain" },
                { 1192, "rain" },
                { 1195, "rain" },
                { 1066, "snow" },
                { 1087, "thunderstorms-day" },
                { 1273, "thunderstorms-day-rain" },
                // ... dan lainnya
            };
        }

        // Fungsi untuk load icon cuaca
        public static Image LoadWeatherIcon(int weatherCode, bool isDay)

```

```

{
    try
    {
        string iconName = GetIconName(weatherCode, isDay);
        string iconPath = Path.Combine(iconFolder, $"{iconName}.png");

        if (File.Exists(iconPath))
        {
            using (var originalImage = Image.FromFile(iconPath))
            {
                return new Bitmap(originalImage);
            }
        }
        return null!;
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine($"Error loading icon:
{ex.Message}");
        return null!;
    }
}

// Fungsi untuk mendapatkan nama icon
private static string GetIconName(int weatherCode, bool isDay)
{
    string iconName;

    if (!weatherCodeToIcon.TryGetValue(weatherCode, out iconName!))
    {
        iconName = isDay ? "clear-day" : "clear-night";
    }

    // Ganti icon siang dengan malam
    if (!isDay)
    {
        if (weatherCode == 1000)
            iconName = "clear-night";
        else if (weatherCode == 1003)
            iconName = "partly-cloudy-night";
        else if (iconName.Contains("-day"))
            iconName = iconName.Replace("-day", "-night");
    }

    return iconName;
}
}

```

PENJELASAN KODE PROGRAM

A. Struktur Utama Form1.cs

1. Deklarasi Konstanta dan Variable

```
private const string API_KEY = "xxxxxxxxxxxxxxxxxx";  
private const string FORECAST_API_URL =  
"http://api.weatherapi.com/v1/forecast.json";  
private HttpClient httpClient;
```

- API_KEY - Kunci autentikasi untuk mengakses WeatherAPI
- FORECAST_API_URL - URL endpoint API untuk data forecast
- httpClient - Instance untuk melakukan HTTP request

2. Constructor dan Inisialisasi

```
public Form1()  
{  
    InitializeComponent();  
    httpClient = new HttpClient();  
    string iconFolder = Path.Combine(Application.StartupPath, "icons");  
    WeatherIcon.IconFolder(iconFolder);  
    InitializeWeatherIcons();  
}
```

Proses:

1. Inisialisasi komponen form
2. Membuat HttpClient baru
3. Menentukan folder icon
4. Menginisialisasi PictureBox untuk icon cuaca

3. Event Handler Button Search

```
private async void btnSearch_Click(object sender, EventArgs e)  
{  
    string city = txtCity.Text.Trim();  
  
    if (string.IsNullOrEmpty(city))  
    {  
        MessageBox.Show("Please enter a city name!", "Warning",  
            MessageBoxButtons.OK, MessageBoxIcon.Warning);  
        return;  
    }  
  
    btnSearch.Enabled = false;  
    btnSearch.Text = "Loading...";  
    await GetWeatherData(city);  
  
    btnSearch.Enabled = true;  
    btnSearch.Text = "Search";  
}
```

Alur:

1. Ambil input nama kota dari TextBox

2. Validasi input tidak kosong
3. Disable button dan ubah teks jadi "Loading..."
4. Panggil fungsi GetWeatherData (async)
5. Enable kembali button setelah selesai

4. Fungsi GetWeatherData (Async)

```
private async Task GetWeatherData(string city)
{
    try
    {
        string url =
        $"{FORECAST_API_URL}?key={API_KEY}&q={city}&days=5&aqi=yes&alerts=yes";
        HttpResponseMessage response = await httpClient.GetAsync(url);

        if (response.IsSuccessStatusCode)
        {
            string json = await response.Content.ReadAsStringAsync();
            Root weatherData = JsonConvert.DeserializeObject<Root>(json)!;
            DisplayWeatherData(weatherData);
        }
        else
        {
            ClearWeatherDisplay();
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Error: {ex.Message}", "Error",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

Proses:

1. Buat URL dengan parameter (API key, city, days, aqi)
2. Kirim HTTP GET request ke API
3. Cek apakah response sukses (status 200)
4. Baca content response sebagai string JSON
5. Deserialize JSON menjadi object Root
6. Tampilkan data ke UI

5. Fungsi DisplayWeatherData

```
private void DisplayWeatherData(Root data)
{
    if (data == null) return;

    lblCity.Text = data.location.name;
    lblCountry.Text = $"{data.location.region}, {data.location.country}";
    lblLastUpdate.Text = $"Last updated: \n{data.current.last_updated}";

    lblTemp.Text = $"{data.current.temp_c}°C";
}
```

```

lblCondition.Text = data.current.condition.text;
lblFeelsLike.Text = $"Feels like: {data.current.feelslike_c}°C";

lblUVValue.Text = data.current.uv.ToString();
lblHumidityValue.Text = $"{data.current.humidity}%";
lblPressureValue.Text = $"{data.current.pressure_mb} mb";
lblCloudValue.Text = $"{data.current.cloud}%";
lblWindValue.Text = $"{data.current.wind_kph} km/h\n{data.current.wind_dir}";
lblPrecipValue.Text = $"{data.current.precip_mm} mm";

if (data.current.air_quality != null)
{
    lblAQValue.Text = $"Air Quality
\n{GetAQIDescription(data.current.air_quality.usepindex)}";
    lblPM25Value.Text = $"PM 2.5: {data.current.air_quality.pm2_5:F1} µg/m³";
    lblPM10Value.Text = $"PM10: {data.current.air_quality.pm10:F1} µg/m³";
    lblCOValue.Text = $"CO: {data.current.air_quality.co:F1} µg/m³";
    lblO3Value.Text = $"O3: {data.current.air_quality.o3:F1} µg/m³";
}

LocalIcon(data.current.condition.code, data.current.is_day == 1,
pictureBoxWeather);

if (data.forecast.forecastday != null && data.forecast.forecastday.Count >=
3)
{
    DisplayForecastDay(data.forecast.forecastday[0], lblDay1Date,
lblDay1Temp, lblDay1Condition, picDay1);
    DisplayForecastDay(data.forecast.forecastday[1], lblDay2Date,
lblDay2Temp, lblDay2Condition, picDay2);
    DisplayForecastDay(data.forecast.forecastday[2], lblDay3Date,
lblDay3Temp, lblDay3Condition, picDay3);
}
}

```

Fungsi:

- Mengambil data dari object Root dan menampilkannya ke Label/PictureBox
- Menggunakan string interpolation untuk format teks
- Memanggil LocalIcon() untuk memuat icon cuaca
- Menampilkan forecast 3 hari dengan looping

B. Class WeatherIcon.cs

1. Dictionary Mapping

```

private static Dictionary<int, string> weatherCodeToIcon = new Dictionary<int,
string>();
{
    { 1000, "clear-day" },
    { 1003, "partly-cloudy-day" },
    { 1006, "cloudy" },
    { 1009, "overcast" },
    { 1030, "mist" },
    { 1135, "fog" },
    { 1147, "fog" },
    { 1063, "drizzle" },
    { 1150, "drizzle" },
    { 1153, "drizzle" },
    { 1168, "sleet" },
    { 1171, "sleet" },
    { 1180, "rain" },
    { 1183, "rain" },
}

```

```

{ 1186, "rain" },
{ 1189, "rain" },
{ 1192, "rain" },
{ 1195, "rain" },
{ 1198, "sleet" },
{ 1201, "sleet" },
{ 1240, "rain" },
{ 1243, "rain" },
{ 1246, "rain" },
{ 1066, "snow" },
{ 1069, "sleet" },
{ 1072, "sleet" },
{ 1114, "snow" },
{ 1117, "snow" },
{ 1204, "sleet" },
{ 1207, "sleet" },
{ 1210, "snow" },
{ 1213, "snow" },
{ 1216, "snow" },
{ 1219, "snow" },
{ 1222, "snow" },
{ 1225, "snow" },
{ 1237, "hail" },
{ 1249, "sleet" },
{ 1252, "sleet" },
{ 1255, "snow" },
{ 1258, "snow" },
{ 1261, "hail" },
{ 1264, "hail" },
{ 1087, "thunderstorms-day" },
{ 1273, "thunderstorms-day-rain" },
{ 1276, "thunderstorms-rain" },
{ 1279, "thunderstorms-day-snow" },
{ 1282, "thunderstorms-snow" }
};

```

Konsep:

- Menyimpan pasangan key-value (kode cuaca → nama icon)
- Memudahkan pencarian icon berdasarkan kode

2. Fungsi LoadWeatherIcon

```

public static Image LoadWeatherIcon(int weatherCode, bool isDay)
{
    try
    {
        string iconName = GetIconName(weatherCode, isDay);
        string iconPath = Path.Combine(iconFolder, $"{iconName}.png");

        if (File.Exists(iconPath))
        {
            using (var originalImage = Image.FromFile(iconPath))
            {
                return new Bitmap(originalImage);
            }
        }
        else
        {
            System.Diagnostics.Debug.WriteLine($"Icon not found: {iconPath}");
        }
    }
}

```



```

        return null!;
    }
}
catch (Exception ex)
{
    System.Diagnostics.Debug.WriteLine($"Error loading icon: {ex.Message}");
    return null!;
}
}

```

Proses:

1. Dapatkan nama icon dari kode cuaca
2. Gabungkan path folder dengan nama file
3. Cek apakah file ada
4. Load image dan return sebagai Bitmap

3. Fungsi GetIconName (Private)

```

    string iconName;

    if (!weatherCodeToIcon.TryGetValue(weatherCode, out iconName!))
    {
        System.Diagnostics.Debug.WriteLine($"⚠ Weather code
{weatherCode} NOT FOUND in mapping! Using default.");
        iconName = isDay ? "clear-day" : "clear-night";
    }
    else
    {
        System.Diagnostics.Debug.WriteLine($"✓ Weather code {weatherCode}
mapped to: {iconName}");
    }

    if (!isDay)
    {
        if (weatherCode == 1000)
        {
            iconName = "clear-night";
        }
        else if (weatherCode == 1003)
        {
            iconName = "partly-cloudy-night";
        }
        else if (weatherCode == 1087)
        {
            iconName = "thunderstorms-night";
        }
        else if (weatherCode == 1273)
        {
            iconName = "thunderstorms-night-rain";
        }
        else if (weatherCode == 1279)
        {
            iconName = "thunderstorms-night-snow";
        }
        else if (iconName.Contains("-day"))
        {
            "

```

```

        iconName = iconName.Replace("-day", "-night");
    }
}

return iconName;
}

```

Logika:

- Cari nama icon dari dictionary
- Jika tidak ketemu, gunakan default (clear-day/clear-night)
- Jika waktu malam, ganti icon siang dengan malam
- Return nama icon yang sudah disesuaikan

C. Class Root.cs - Data Models

1. Konsep Required Property

```

public class Current
{
    public required string last_updated { get; set; }
    public required object temp_c { get; set; }
    Condition condition { get; set; }
}

```

Penjelasan:

- required - Property wajib diisi saat membuat object
- Mencegah null reference exception
- Compiler akan warning jika property tidak diisi

2. JSON Property Mapping

```

[JsonProperty("us-epa-index")]
public required int usepaindex { get; set; }

```

Fungsi:

- Memetakan property C# ke key JSON yang berbeda
- JSON key: "us-epa-index" (ada strip)
- C# property: usepaindex (tanpa strip)

3. Nested Objects

```

public class Root
{
    public required Location location { get; set; }
    public required Current current { get; set; }
    public required Forecast forecast { get; set; }
}

```

Struktur:

- Root sebagai container utama
- Berisi object Location, Current, Forecast

- Mencerminkan struktur JSON dari API

FITUR-FITUR KHUSUS

1. Icon Dinamis Siang/Malam

Program secara otomatis mengganti icon berdasarkan waktu:

- `is_day = 1` (Siang) → Gunakan icon "-day"
- `is_day = 0` (Malam) → Gunakan icon "-night"

```
LocalIcon(data.current.condition.code, data.current.is_day == 1,
pictureBoxWeather);
```

2. Icon Alert Kondisional

Icon berubah berdasarkan nilai parameter:

Tekanan Udara:

```
if (data.current.pressure_mb >= 1013.25)
{
    picPressure.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"pressure-high.png"));
}
else
{
    picPressure.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"pressure-low.png"));
}
```

Angin:

```
if (data.current.wind_kph >= 21)
{
    picWind.LoadAsync(Path.Combine(Application.StartupPath, "icons", "wind-
alert.png"));
}
else
{
    picWind.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"wind.png"));
}
```

Curah Hujan:

```
if (data.current.precip_mm >= 11)
{
    picPrecip.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"raindrops.png"));
}
else
{
    picPrecip.LoadAsync(Path.Combine(Application.StartupPath, "icons",
"raindrop.png"));
}
```

3. Indeks UV Dinamis

```
int uvIndex = (int)Math.Floor(data.current.uv);
uvIndex = Math.Min(uvIndex, 11); // Cap maksimal di 11

string iconName = uvIndex >= 1
    ? $"uv-index-{uvIndex}.png"
    : "not-available.png";

picUV.LoadAsync(Path.Combine(Application.StartupPath, "icons", iconName));
```

4. Kualitas Udara dengan Deskripsi

```
private string GetAQIDescription(int index)
{
    return index switch
    {
        1 => "Good",
        2 => "Moderate",
        3 => "Unhealthy for Sensitive Groups",
        4 => "Unhealthy",
        5 => "Very Unhealthy",
        6 => "Hazardous",
        _ => "Not Available"
    };
}
```

5. Format Tanggal Forecast

```
DateTime date = DateTime.Parse(forecastDay.date);
lblDate.Text = date.ToString("ddd, MMM dd", CultureInfo.InvariantCulture);
Output: Mon, Jan 15
```

6. Enter Key untuk Search

```
private void txtCity_KeyPress(object sender, KeyPressEventArgs e)
{
    if (e.KeyChar == (char)Keys.Enter)
    {
        btnSearch_Click(sender, e);
        e.Handled = true;
    }
}
```

User bisa tekan Enter di TextBox untuk langsung search.

7. Resource Management

```
protected override void OnFormClosing(FormClosingEventArgs e)
{
    httpClient?.Dispose();
    base.OnFormClosing(e);
}
```

Dispose HttpClient saat form ditutup untuk membebaskan memory.

LAMPIRAN

Kode Program Lengkap

1. Form1.cs

```
59         picDay3 = CreateForecastIcon(panelDay3);
60     }
61
62     // Fungsi helper untuk membuat PictureBox icon forecast di dalam panel
63     private PictureBox CreateForecastIcon(Panel panel)
64     {
65         var pictureBox = new PictureBox();
66         pictureBox.Location = new Point(53, 23); // Posisi di dalam panel
67         pictureBox.Size = new Size(80, 80); // Ukuran 76x71 pixel
68         pictureBox.SizeMode = PictureBoxSizeMode.Zoom; // Mode zoom
69         pictureBox.BackColor = Color.Transparent; // Background transparan
70         panel.Controls.Add(pictureBox); // Tambahkan ke panel
71         panel.BringToFront(); // Bawa panel ke depan
72         return pictureBox;
73     }
74
75     // Fungsi ketika tombol Search diklik
76     private async void btnSearch_Click(object sender, EventArgs e)
77     {
78         // Ambil teks dari textbox dan hapus spasi di awal/akhir
79         string city = txtCity.Text.Trim();
80
81         // Validasi: cek apakah nama kota kosong
82         if (string.IsNullOrEmpty(city))
83         {
84             MessageBox.Show("Please enter a city name!", "Warning",
85                 MessageBoxButtons.OK, MessageBoxIcon.Warning);
86             return;
87         }
88
89         // Disable tombol search agar tidak bisa diklik saat loading
90         btnSearch.Enabled = false;
91         btnSearch.Text = "Loading..."; // Ubah teks tombol menjadi "Loading..."
92         lblStatus.Text = "Fetching weather data..."; // Update status
93         lblStatus.ForeColor = Color.White; // Warna teks putih
94
95         // Panggil Fungsi async untuk mendapatkan data cuaca
96         await GetWeatherData(city);
97
98         // Enable kembali tombol search setelah selesai
99         btnSearch.Enabled = true;
100        btnSearch.Text = "Search"; // Kembalikan teks tombol
101    }
102
103    // Fungsi async untuk mengambil data cuaca dari API
104    private async Task GetWeatherData(string city)
105    {
106        try
107        {
108            // Buat URL dengan parameter: API key, nama kota, jumlah hari forecast, dan AQI
109            string url = $"{FORECAST_API_URL}?key={API_KEY}&q={city}&days=5&aqi=yes&alerts=yes";
110            // Kirim request GET ke API
111            HttpResponseMessage response = await httpClient.GetAsync(url);
112
113            // Cek apakah response berhasil (status code 200)
114            if (response.IsSuccessStatusCode)
115            {
116                // Baca content response sebagai string JSON
117                string json = await response.Content.ReadAsStringAsync();
118                // Deserialize JSON menjadi object Root
119                Root weatherData = JsonConvert.DeserializeObject<Root>(json);
120                // Tampilkan data cuaca di UI
121                DisplayWeatherData(weatherData);
122            }
123        }
124        catch { }
```

```

121         DisplayWeatherData(weatherData);
122     }
123     else
124     {
125         // Jika gagal, tampilkan pesan kota tidak ditemukan
126         lblStatus.Text = "City not found!";
127         lblStatus.ForeColor = Color.FromArgb(255, 100, 100); // Warna merah
128         // Bersihkan tampilan cuaca
129         ClearWeatherDisplay();
130     }
131 }
132 catch (Exception ex)
133 {
134     // Tangkap error dan tampilkan pesan error
135     MessageBox.Show($"Error: {ex.Message}", "Error",
136         MessageBoxButtons.OK, MessageBoxIcon.Error);
137     lblStatus.Text = "Error loading data";
138     lblStatus.ForeColor = Color.FromArgb(255, 100, 100);
139 }
140 }
141
142 // Fungsi untuk menampilkan data cuaca ke UI
143 private void DisplayWeatherData(Root data)
144 {
145     // Validasi: cek apakah data null
146     if (data == null) return;
147
148     // Tampilkan data lokasi
149     lblCity.Text = data.location.name; // Nama kota
150     lblCountry.Text = $"{data.location.region}, {data.location.country}"; // Region dan negara
151     lblLastUpdate.Text = $"Last updated: \n{data.current.last_updated}"; // Waktu update terakhir
152
153     // Tampilkan data cuaca saat ini
154     lblTemp.Text = $"{data.current.temp_c}°C"; // Suhu dalam Celsius
155     lblCondition.Text = data.current.condition.text; // Kondisi cuaca (text)
156     lblFeelslike.Text = $"Feels like: {data.current.feelslike_c}°C"; // Suhu yang dirasakan
157
158     // Tampilkan data detail cuaca
159     lblUVValue.Text = data.current.uv.ToString(); // Indeks UV
160     lblHumidityValue.Text = $"{data.current.humidity}%"; // Kelembaban
161     lblPressureValue.Text = $"{data.current.pressure_mb} mb"; // Tekanan udara
162     lblCloudValue.Text = $"{data.current.cloud}%"; // Persentase awan
163     lblWindValue.Text = $"{data.current.wind_kph} km/h\n{data.current.wind_dir}"; // Kecepatan dan arah angin
164     lblPrecipValue.Text = $"{data.current.precip_mm} mm"; // Curah hujan
165
166     // Tampilkan data kualitas udara jika tersedia
167     if (data.current.air_quality != null)
168     {
169         // Tampilkan indeks kualitas udara dengan deskripsi
170         lblAQValue.Text = $"Air Quality \n{GetAQIDescription(data.current.air_quality.usepindex)}";
171         // Tampilkan PM 2.5 (partikel halus)
172         lblPM25Value.Text = $"PM 2.5: {data.current.air_quality.pm2_5:F1} µg/m³";
173         // Tampilkan PM 10 (partikel kasar)
174         lblPM10Value.Text = $"PM10: {data.current.air_quality.pm10:F1} µg/m³";
175         // Tampilkan Carbon Monoxide
176         lblCOValue.Text = $"CO: {data.current.air_quality.co:F1} µg/m³";
177         // Tampilkan Ozone
178         lblO3Value.Text = $"O3: {data.current.air_quality.o3:F1} µg/m³";
179     }
180
181     // Load icon cuaca utama berdasarkan kode cuaca dan waktu (siang/malam)
182     LocalIcon(data.current.condition.code, data.current.is_day == 1, pictureBoxWeather);
183 }

```

```

184 // Tampilkan forecast 3 hari jika data tersedia
185 if (data.forecast.forecastday != null && data.forecast.forecastday.Count >= 3)
186 {
187     DisplayForecastDay(data.forecast.forecastday[0], lblDay1Date, lblDay1Temp, lblDay1Condition, picDay1);
188     DisplayForecastDay(data.forecast.forecastday[1], lblDay2Date, lblDay2Temp, lblDay2Condition, picDay2);
189     DisplayForecastDay(data.forecast.forecastday[2], lblDay3Date, lblDay3Temp, lblDay3Condition, picDay3);
190 }
191
192 // Load icon kelembaban
193 picHumidity.LoadAsync(Path.Combine(Application.StartupPath, "icons", "humidity.png"));
194
195 // Load icon tekanan udara (tinggi/rendah berdasarkan nilai tekanan standar 1013.25 mb)
196 if (data.current.pressure_mb >= 1013.25)
197 {
198     picPressure.LoadAsync(Path.Combine(Application.StartupPath, "icons", "pressure-high.png"));
199 }
200 else
201 {
202     picPressure.LoadAsync(Path.Combine(Application.StartupPath, "icons", "pressure-low.png"));
203 }
204
205 // Load icon angin (alert jika kecepatan >= 21 km/h)
206 if (data.current.wind_kph >= 21)
207 {
208     picWind.LoadAsync(Path.Combine(Application.StartupPath, "icons", "wind-alert.png"));
209 }
210 else
211 {
212     picWind.LoadAsync(Path.Combine(Application.StartupPath, "icons", "wind.png"));
213 }
214
215 // Load icon curah hujan (banyak tetes jika >= 11 mm)
216 if (data.current.precip_mm >= 11)
217 {
218     picPrecip.LoadAsync(Path.Combine(Application.StartupPath, "icons", "raindrops.png"));
219 }
220 else
221 {
222     picPrecip.LoadAsync(Path.Combine(Application.StartupPath, "icons", "raindrop.png"));
223 }
224
225 // Load icon UV berdasarkan indeks UV (0-11+)
226 int uvIndex = (int)Math.Floor(data.current.uv);
227 uvIndex = Math.Min(uvIndex, 11); // Cap maksimal di 11
228
229 string iconName = uvIndex >= 1
230     ? $"uv-index-{uvIndex}.png"
231     : "not-available.png";
232
233 picUV.LoadAsync(Path.Combine(Application.StartupPath, "icons", iconName));
234
235 // Load icon awan berdasarkan persentase awan
236 if (data.current.cloud <= 50)
237 {
238     picCloud.LoadAsync(Path.Combine(Application.StartupPath, "icons", "mist.png"));
239 }
240 else
241 {
242     picCloud.LoadAsync(Path.Combine(Application.StartupPath, "icons", "overcast.png"));
243 }
244
245 // Update status berhasil dengan warna hijau
246 lblStatus.Text = "Weather data loaded successfully!";

```

```

247     lblStatus.ForeColor = Color.FromArgb(150, 255, 150);
248 }
249
250 // Fungsi untuk menampilkan data forecast per hari
251 private void DisplayForecastDay(ForecastDay forecastDay, Label lblDate, Label lblTemp, Label lblCondition, PictureBox pictureBox)
252 {
253     // Validasi: cek apakah data forecast null
254     if (forecastDay == null) return;
255
256     // Parse tanggal dan format menjadi "Day, Month DD" (contoh: Mon, Jan 15)
257     DateTime date = DateTime.Parse(forecastDay.date);
258     lblDate.Text = date.ToString("ddd, MMM dd", CultureInfo.InvariantCulture);
259
260     // Tampilkan suhu maksimal dan minimal (format: XX°/YY°)
261     lblTemp.Text = $"{forecastDay.day.maxtemp_c:F0}°/{forecastDay.day.mintemp_c:F0}°";
262     // Tampilkan kondisi cuaca
263     lblCondition.Text = forecastDay.day.condition.text;
264
265     // Load icon cuaca dari folder lokal (selalu gunakan icon siang untuk forecast)
266     LocalIcon(forecastDay.day.condition.code, true, pictureBox);
267 }
268
269 // Fungsi untuk memuat icon cuaca dari folder lokal
270 private void LocalIcon(int weatherCode, bool isDay, PictureBox pictureBox)
271 {
272     try
273     {
274         // Load icon menggunakan class WeatherIcon
275         Image icon = WeatherIcon.LoadWeatherIcon(weatherCode, isDay);
276         // Jika icon berhasil dimuat, set ke PictureBox
277         if (icon != null)
278             pictureBox.Image = icon;
279         else
280             pictureBox.Image = null; // Set null jika gagal
281     }
282     catch
283     {
284         // Jika terjadi error, set image null
285         pictureBox.Image = null;
286     }
287 }
288
289 // Fungsi untuk mendapatkan deskripsi kualitas udara berdasarkan indeks
290 private string GetAQIDescription(int index)
291 {
292     return index switch
293     {
294         1 => "Good", // Baik
295         2 => "Moderate", // Sedang
296         3 => "Unhealthy for Sensitive Groups", // Tidak sehat untuk kelompok sensitif
297         4 => "Unhealthy", // Tidak sehat
298         5 => "Very Unhealthy", // Sangat tidak sehat
299         6 => "Hazardous", // Berbahaya
300         _ => "Not Available" // Tidak tersedia
301     };
302 }
303
304 // Fungsi untuk membersihkan/mereset tampilan cuaca
305 private void ClearWeatherDisplay()
306 {
307     // Reset semua label menjadi "-"
308     lblCity.Text = "-";

```



```

309         lblCountry.Text = "-";
310         lblTemp.Text = "-";
311         lblCondition.Text = "-";
312         lblFeelslike.Text = "Feels like: -";
313         lblUVValue.Text = "-";
314         lblHumidityValue.Text = "-";
315         lblPressureValue.Text = "-";
316         lblCloudValue.Text = "-";
317         lblWindValue.Text = "-";
318         lblPrecipValue.Text = "-";
319         lblAQValue.Text = "-";
320         lblPM25Value.Text = "PM 2.5: -";
321         lblPM10Value.Text = "PM10: -";
322         lblCOValue.Text = "CO: -";
323         lblO3Value.Text = "O3: -";
324
325         // Hapus semua gambar icon
326         pictureBoxWeather.Image = null;
327         picDay1.Image = null;
328         picDay2.Image = null;
329         picDay3.Image = null;
330
331         // Reset label forecast 5 hari menjadi "N/A"
332         lblDay1Date.Text = "N/A";
333         lblDay1Temp.Text = "N/A";
334         lblDay1Condition.Text = "N/A";
335
336         lblDay2Date.Text = "N/A";
337         lblDay2Temp.Text = "N/A";
338         lblDay2Condition.Text = "N/A";
339
340         lblDay3Date.Text = "N/A";
341         lblDay3Temp.Text = "N/A";
342         lblDay3Condition.Text = "N/A";
343
344     }
345
346     // Fungsi ketika user menekan tombol di textbox
347     private void txtCity_KeyPress(object sender, KeyPressEventArgs e)
348     {
349         // Cek jika tombol yang ditekan adalah Enter
350         if (e.KeyChar == (char)Keys.Enter)
351         {
352             // Trigger event btnSearch_Click (sama seperti klik tombol search)
353             btnSearch_Click(sender, e);
354             // Set Handled = true agar tidak ada "beep" sound
355             e.Handled = true;
356         }
357     }
358
359     // Override Fungsi OnFormClosing untuk cleanup saat form ditutup
360     protected override void OnFormClosing(FormClosingEventArgs e)
361     {
362         // Dispose HttpClient untuk membebaskan resource
363         httpClient?.Dispose();
364         // Panggil base Fungsi
365         base.OnFormClosing(e);
366     }
367 }
368 }

```

2. Root.cs

```
1 using Newtonsoft.Json;
2 using System.Drawing.Imaging;
3 using WeatherApp;
4
5 namespace WeatherApp
6 {
7     // Class untuk merepresentasikan data kualitas udara
8     public class AirQuality
9     {
10         public required object co { get; set; } // Carbon Monoxide (Karbon Monoksida)
11         public required object no2 { get; set; } // Nitrogen Dioxide (Nitrogen Dioksida)
12         public required object o3 { get; set; } // Ozone (Ozon)
13         public required object so2 { get; set; } // Sulfur Dioxide (Sulfur Dioksida)
14         public required object pm2_5 { get; set; } // Particulate Matter 2.5 (Partikel halus 2.5 mikron)
15         public required object pm10 { get; set; } // Particulate Matter 10 (Partikel 10 mikron)
16
17         // Indeks kualitas udara US EPA dengan mapping JSON property
18         [JsonProperty("us-epa-index")]
19         public required int usepaindex { get; set; }
20
21         // Indeks kualitas udara UK DEFRA dengan mapping JSON property
22         [JsonProperty("gb-defra-index")]
23         public required int gbdefraindex { get; set; }
24     }
25
26     // Class untuk merepresentasikan kondisi cuaca
27     public class Condition
28     {
29         public required string text { get; set; } // Deskripsi kondisi cuaca dalam teks (contoh: "Sunny", "Rainy")
30         public required string icon { get; set; } // URL icon cuaca dari API
31         public required int code { get; set; } // Kode numerik kondisi cuaca (untuk mapping icon lokal)
32     }
33
34     // Class untuk merepresentasikan data cuaca saat ini (current weather)
35     public class Current
36     {
37         public required object last_updated_epoch { get; set; } // Waktu update terakhir dalam format epoch/unix timestamp
38         public required string last_updated { get; set; } // Waktu update terakhir dalam format string readable
39         public required object temp_c { get; set; } // Suhu dalam Celsius
40         public required object temp_f { get; set; } // Suhu dalam Fahrenheit
41         public required int is_day { get; set; } // Indikator siang/malam (1 = siang, 0 = malam)
42         public required Condition condition { get; set; } // Object kondisi cuaca
43         public required object wind_mph { get; set; } // Kecepatan angin dalam miles per hour
44         public required double wind_kph { get; set; } // Kecepatan angin dalam kilometer per hour
45         public required object wind_degree { get; set; } // Arah angin dalam derajat
46         public required string wind_dir { get; set; } // Arah angin dalam teks (contoh: "N", "SE", "SW")
47         public required double pressure_mb { get; set; } // Tekanan udara dalam millibar
48         public required double pressure_in { get; set; } // Tekanan udara dalam inches
49         public required double precip_mm { get; set; } // Curah hujan dalam millimeter
50         public required double precip_in { get; set; } // Curah hujan dalam inches
51         public required double humidity { get; set; } // Kelembaban udara dalam persen
52         public required double cloud { get; set; } // Persentase tutupan awan
53         public required double feelslike_c { get; set; } // Suhu yang dirasakan dalam Celsius
54         public required double feelslike_f { get; set; } // Suhu yang dirasakan dalam Fahrenheit
55         public required double vis_km { get; set; } // Jarak pandang/visibility dalam kilometer
56         public required double vis_miles { get; set; } // Jarak pandang/visibility dalam miles
57         public required double uv { get; set; } // Indeks UV (ultraviolet)
58         public required double gust_mph { get; set; } // Kecepatan hembusan angin maksimal dalam miles per hour
59         public required object gust_kph { get; set; } // Kecepatan hembusan angin maksimal dalam kilometer per hour
60         public required AirQuality air_quality { get; set; } // Object data kualitas udara
```

```

61 }
62
63 // Class untuk merepresentasikan data lokasi
64 public class Location
65 {
66     public required string name { get; set; } // Nama kota/lokasi
67     public required string region { get; set; } // Nama region/provinsi/state
68     public required string country { get; set; } // Nama negara
69     public required object lat { get; set; } // Koordinat latitude (garis lintang)
70     public required object lon { get; set; } // Koordinat longitude (garis bujur)
71     public required string tz_id { get; set; } // Timezone ID (contoh: "Asia/Jakarta")
72     public required object localtime_epoch { get; set; } // Waktu lokal dalam format epoch
73     public required string localtime { get; set; } // Waktu lokal dalam format string
74 }
75
76 // Class untuk merepresentasikan data forecast/ramalan cuaca
77 public class Forecast
78 {
79     public required List<Forecastday> forecastday { get; set; } // List/daftar ramalan cuaca per hari
80 }
81
82 // Class untuk merepresentasikan data ramalan cuaca per hari
83 public class Forecastday
84 {
85     public required string date { get; set; } // Tanggal ramalan dalam format string (YYYY-MM-DD)
86     public required int date_epoch { get; set; } // Tanggal ramalan dalam format epoch
87     public required Day day { get; set; } // Object data cuaca untuk hari tersebut
88 }
89
90 // Class untuk merepresentasikan detail cuaca harian
91 public class Day
92 {
93     public required double maxtemp_c { get; set; } // Suhu maksimal dalam Celsius
94     public required double maxtemp_f { get; set; } // Suhu maksimal dalam Fahrenheit
95     public required double mintemp_c { get; set; } // Suhu minimal dalam Celsius
96     public required double mintemp_f { get; set; } // Suhu minimal dalam Fahrenheit
97     public required double avgtemp_c { get; set; } // Suhu rata-rata dalam Celsius
98     public required double avgtemp_f { get; set; } // Suhu rata-rata dalam Fahrenheit
99     public required double maxwind_mph { get; set; } // Kecepatan angin maksimal dalam miles per hour
100    public required double maxwind_kph { get; set; } // Kecepatan angin maksimal dalam kilometer per hour
101    public required double totalprecip_mm { get; set; } // Total curah hujan dalam millimeter
102    public required double totalprecip_in { get; set; } // Total curah hujan dalam inches
103    public required double avgvis_km { get; set; } // Rata-rata jarak pandang dalam kilometer
104    public required double avgvis_miles { get; set; } // Rata-rata jarak pandang dalam miles
105    public required int avghumidity { get; set; } // Rata-rata kelembaban dalam persen
106    public required int daily_will_it_rain { get; set; } // Indikator apakah akan hujan (1 = ya, 0 = tidak)
107    public required int daily_chance_of_rain { get; set; } // Persentase kemungkinan hujan
108    public required int daily_will_it_snow { get; set; } // Indikator apakah akan turun salju (1 = ya, 0 = tidak)
109    public required int daily_chance_of_snow { get; set; } // Persentase kemungkinan salju
110    public required Condition condition { get; set; } // Object kondisi cuaca
111    public required double uv { get; set; } // Indeks UV
112 }
113
114 // Class Root sebagai container utama untuk semua data response dari API
115 public class Root
116 {
117     public required Location location { get; set; } // Data lokasi
118     public required Current current { get; set; } // Data cuaca saat ini
119     public required Forecast forecast { get; set; } // Data ramalan cuaca
120     public required object alert { get; set; } // Data alert/peringatan cuaca (jika ada)
121 }
122 }

```

3. Form1.Designer.cs

```
1 namespace WeatherApp
2 {
3     partial class Form1
4     {
5         private System.ComponentModel.IContainer components = null;
6         private System.Windows.Forms.TextBox txtCity;
7         private System.Windows.Forms.Button btnSearch;
8         private System.Windows.Forms.Label lblTitle;
9         private System.Windows.Forms.Label lblCity;
10        private System.Windows.Forms.Label lblCountry;
11        private System.Windows.Forms.Label lblTemp;
12        private System.Windows.Forms.Label lblCondition;
13        private System.Windows.Forms.Label lblFeelsLike;
14        // Card Labels
15        private System.Windows.Forms.Label lblUVValue;
16        private System.Windows.Forms.Label lblHumidityValue;
17        private System.Windows.Forms.Label lblPressureValue;
18        private System.Windows.Forms.Label lblCloudValue;
19        private System.Windows.Forms.Label lblWindValue;
20        private System.Windows.Forms.Label lblPrecipValue;
21        private System.Windows.Forms.Label lblAQValue;
22        private System.Windows.Forms.Label lblPM25Value;
23        private System.Windows.Forms.Label lblPM10Value;
24        private System.Windows.Forms.Label lblCOValue;
25        private System.Windows.Forms.Label lblO3Value;
26        private System.Windows.Forms.Label lblStatus;
27
28        // Forecast day panels
29        private System.Windows.Forms.Panel panelDay1;
30        private System.Windows.Forms.Panel panelDay2;
31        private System.Windows.Forms.Panel panelDay3;
32
33        // Forecast labels
34        private System.Windows.Forms.Label lblDay1Date;
35        private System.Windows.Forms.Label lblDay1Temp;
36        private System.Windows.Forms.Label lblDay1Condition;
37
38        private System.Windows.Forms.Label lblDay2Date;
39        private System.Windows.Forms.Label lblDay2Temp;
40        private System.Windows.Forms.Label lblDay2Condition;
41
42        private System.Windows.Forms.Label lblDay3Date;
43        private System.Windows.Forms.Label lblDay3Temp;
44        private System.Windows.Forms.Label lblDay3Condition;
45
46        // Panels for cards
47        private System.Windows.Forms.Panel panelForecast;
48        private System.Windows.Forms.Panel panelUV;
49        private System.Windows.Forms.Panel panelHumidity;
50        private System.Windows.Forms.Panel panelPressure;
51        private System.Windows.Forms.Panel panelCloud;
52        private System.Windows.Forms.Panel panelWind;
53        private System.Windows.Forms.Panel panelPrecip;
54        private System.Windows.Forms.Panel panelAirQuality;
55        private System.Windows.Forms.Panel panelAirDetails;
56
57        protected override void Dispose(bool disposing)
58        {
59            if (disposing && (components != null))
60            {
61                components.Dispose();
62            }
63            base.Dispose(disposing);
64        }
65
66        private void InitializeComponent()
67        {
68            System.ComponentModel.ComponentResourceManager resources = new System.ComponentModel.ComponentResourceManager(typeof(Form1));
69            txtCity = new TextBox();
70            btnSearch = new Button();
```

```

71     lblTitle = new Label();
72     lblCity = new Label();
73     lblCountry = new Label();
74     lblTemp = new Label();
75     lblCondition = new Label();
76     lblFeelsLike = new Label();
77     lblUVValue = new Label();
78     lblHumidityValue = new Label();
79     lblPressureValue = new Label();
80     lblCloudValue = new Label();
81     lblWindValue = new Label();
82     lblPrecipValue = new Label();
83     lblAQValue = new Label();
84     lblPM25Value = new Label();
85     lblPM10Value = new Label();
86     lblCOValue = new Label();
87     lblO3Value = new Label();
88     lblStatus = new Label();
89     panelDay1 = new Panel();
90     lblDay1Date = new Label();
91     lblDay1Temp = new Label();
92     lblDay1Condition = new Label();
93     panelDay2 = new Panel();
94     lblDay2Date = new Label();
95     lblDay2Temp = new Label();
96     lblDay2Condition = new Label();
97     panelDay3 = new Panel();
98     lblDay3Date = new Label();
99     lblDay3Temp = new Label();
100    lblDay3Condition = new Label();
101    panelForecast = new Panel();
102    lblLastUpdate = new Label();
103    panelUV = new Panel();
104    picUV = new PictureBox();
105    lblTitleUV = new Label();
106    panelHumidity = new Panel();
107    picHumidity = new PictureBox();
108    lblTitleHumidity = new Label();
109    panelPressure = new Panel();
110    picPressure = new PictureBox();
111    lblTitlePressure = new Label();
112    panelCloud = new Panel();
113    picCloud = new PictureBox();
114    lblTitleCloud = new Label();
115    panelWind = new Panel();
116    picWind = new PictureBox();
117    lblTitleWind = new Label();
118    panelPrecip = new Panel();
119    picPrecip = new PictureBox();
120    lblTitlePrecip = new Label();
121    panelAirQuality = new Panel();
122    panelAirDetails = new Panel();
123    panelDay1.SuspendLayout();
124    panelDay2.SuspendLayout();
125    panelDay3.SuspendLayout();
126    panelForecast.SuspendLayout();
127    panelUV.SuspendLayout();
128    ((System.ComponentModel.ISupportInitialize)picUV).BeginInit();
129    panelHumidity.SuspendLayout();
130    ((System.ComponentModel.ISupportInitialize)picHumidity).BeginInit();
131    panelPressure.SuspendLayout();
132    ((System.ComponentModel.ISupportInitialize)picPressure).BeginInit();
133    panelCloud.SuspendLayout();
134    ((System.ComponentModel.ISupportInitialize)picCloud).BeginInit();
135    panelWind.SuspendLayout();
136    ((System.ComponentModel.ISupportInitialize)picWind).BeginInit();
137    panelPrecip.SuspendLayout();
138    ((System.ComponentModel.ISupportInitialize)picPrecip).BeginInit();
139    panelAirQuality.SuspendLayout();
140    panelAirDetails.SuspendLayout();
141    SuspendLayout();
142    //
143    // txtCity
144    //
145    txtCity.Focus();

```

```

145         txtCity.Font = new Font("Segoe UI", 12F);
146         txtCity.Location = new Point(29, 60);
147         txtCity.Margin = new Padding(3, 4, 3, 4);
148         txtCity.Name = "txtCity";
149         txtCity.Size = new Size(491, 34);
150         txtCity.TabIndex = 1;
151         txtCity.KeyPress += txtCity_KeyPress;
152         //
153         // btnSearch
154         //
155         btnSearch.BackColor = Color.Transparent;
156         btnSearch.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
157         btnSearch.BackgroundImageLayout = ImageLayout.Stretch;
158         btnSearch.Cursor = Cursors.Hand;
159         btnSearch.FlatAppearance.BorderSize = 0;
160         btnSearch.FlatStyle = FlatStyle.Flat;
161         btnSearch.Font = new Font("Segoe UI", 11F, FontStyle.Bold);
162         btnSearch.ForeColor = Color.White;
163         btnSearch.Location = new Point(526, 55);
164         btnSearch.Margin = new Padding(3, 4, 3, 4);
165         btnSearch.Name = "btnSearch";
166         btnSearch.Size = new Size(137, 39);
167         btnSearch.TabIndex = 2;
168         btnSearch.Text = "Search";
169         btnSearch.UseVisualStyleBackColor = false;
170         btnSearch.Click += btnSearch_Click;
171         //
172         // lblTitle
173         //
174         lblTitle.AutoSize = true;
175         lblTitle.BackColor = Color.Transparent;
176         lblTitle.Font = new Font("Segoe UI", 18F, FontStyle.Bold);
177         lblTitle.ForeColor = Color.White;
178         lblTitle.Location = new Point(29, 15);
179         lblTitle.Name = "lblTitle";
180         lblTitle.Size = new Size(160, 41);
181         lblTitle.TabIndex = 0;
182         lblTitle.Text = "WEATHER";
183         //
184         // lblCity
185         //
186         lblCity.AutoSize = true;
187         lblCity.Font = new Font("Segoe UI", 20F, FontStyle.Bold);
188         lblCity.ForeColor = Color.White;
189         lblCity.Location = new Point(197, 47);
190         lblCity.Name = "lblCity";
191         lblCity.Size = new Size(34, 46);
192         lblCity.TabIndex = 1;
193         lblCity.Text = "-";
194         //
195         // lblCountry
196         //
197         lblCountry.AutoSize = true;
198         lblCountry.Font = new Font("Segoe UI", 11F);
199         lblCountry.ForeColor = Color.FromArgb(220, 230, 240);
200         lblCountry.Location = new Point(197, 93);
201         lblCountry.Name = "lblCountry";
202         lblCountry.Size = new Size(20, 25);
203         lblCountry.TabIndex = 2;
204         lblCountry.Text = "-";
205         //
206         // lblTemp
207         //
208         lblTemp.AutoSize = true;
209         lblTemp.Font = new Font("Segoe UI", 48F, FontStyle.Bold);
210         lblTemp.ForeColor = Color.White;
211         lblTemp.Location = new Point(183, 127);
212         lblTemp.Name = "lblTemp";
213         lblTemp.Size = new Size(77, 106);
214         lblTemp.TabIndex = 3;
215         lblTemp.Text = "-";
216         //
217         // lblCondition
218         //
219         lblCondition.AutoSize = true;

```



```

220     lblCondition.Font = new Font("Segoe UI", 10.8F, FontStyle.Regular, GraphicsUnit.Point, 0);
221     lblCondition.ForeColor = Color.FromArgb(220, 230, 240);
222     lblCondition.Location = new Point(447, 188);
223     lblCondition.Name = "lblCondition";
224     lblCondition.Size = new Size(19, 25);
225     lblCondition.TabIndex = 4;
226     lblCondition.Text = "-";
227     //
228     // lblFeelsLike
229     //
230     lblFeelsLike.AutoSize = true;
231     lblFeelsLike.Font = new Font("Segoe UI", 10F);
232     lblFeelsLike.ForeColor = Color.FromArgb(200, 220, 230);
233     lblFeelsLike.Location = new Point(197, 245);
234     lblFeelsLike.Name = "lblFeelsLike";
235     lblFeelsLike.Size = new Size(93, 23);
236     lblFeelsLike.TabIndex = 5;
237     lblFeelsLike.Text = "Feels like: -";
238     //
239     // lblUVValue
240     //
241     lblUVValue.Font = new Font("Segoe UI", 28F, FontStyle.Bold);
242     lblUVValue.ForeColor = Color.White;
243     lblUVValue.Location = new Point(0, 55);
244     lblUVValue.Name = "lblUVValue";
245     lblUVValue.Size = new Size(169, 67);
246     lblUVValue.TabIndex = 1;
247     lblUVValue.Text = "-";
248     lblUVValue.TextAlign = ContentAlignment.MiddleLeft;
249     //
250     // lblHumidityValue
251     //
252     lblHumidityValue.Font = new Font("Segoe UI", 28F, FontStyle.Bold);
253     lblHumidityValue.ForeColor = Color.White;
254     lblHumidityValue.Location = new Point(0, 55);
255     lblHumidityValue.Name = "lblHumidityValue";
256     lblHumidityValue.Size = new Size(180, 67);
257     lblHumidityValue.TabIndex = 1;
258     lblHumidityValue.Text = "-";
259     lblHumidityValue.TextAlign = ContentAlignment.MiddleLeft;
260     //
261     // lblPressureValue
262     //
263     lblPressureValue.Font = new Font("Segoe UI", 20F, FontStyle.Bold);
264     lblPressureValue.ForeColor = Color.White;
265     lblPressureValue.Location = new Point(0, 67);
266     lblPressureValue.Name = "lblPressureValue";
267     lblPressureValue.Size = new Size(169, 67);
268     lblPressureValue.TabIndex = 1;
269     lblPressureValue.Text = "-";
270     lblPressureValue.TextAlign = ContentAlignment.MiddleLeft;
271     //
272     // lblCloudValue
273     //
274     lblCloudValue.Font = new Font("Segoe UI", 28F, FontStyle.Bold);
275     lblCloudValue.ForeColor = Color.White;
276     lblCloudValue.Location = new Point(0, 67);
277     lblCloudValue.Name = "lblCloudValue";
278     lblCloudValue.Size = new Size(180, 67);
279     lblCloudValue.TabIndex = 1;
280     lblCloudValue.Text = "-";
281     lblCloudValue.TextAlign = ContentAlignment.MiddleLeft;
282     //
283     // lblWindValue
284     //
285     lblWindValue.Font = new Font("Segoe UI", 20F, FontStyle.Bold);
286     lblWindValue.ForeColor = Color.White;
287     lblWindValue.Location = new Point(0, 55);
288     lblWindValue.Name = "lblWindValue";
289     lblWindValue.Size = new Size(169, 121);
290     lblWindValue.TabIndex = 1;
291     lblWindValue.Text = "-";
292     //
293     // lblPrecipValue
294     //

```

```

294 //
295 lblPrecipValue.Font = new Font("Segoe UI", 20F, FontStyle.Bold);
296 lblPrecipValue.ForeColor = Color.White;
297 lblPrecipValue.Location = new Point(0, 55);
298 lblPrecipValue.Name = "lblPrecipValue";
299 lblPrecipValue.Size = new Size(180, 67);
300 lblPrecipValue.TabIndex = 1;
301 lblPrecipValue.Text = "-";
302 lblPrecipValue.TextAlign = ContentAlignment.MiddleLeft;
303 //
304 // lblAQValue
305 //
306 lblAQValue.Font = new Font("Segoe UI", 16F, FontStyle.Bold);
307 lblAQValue.ForeColor = Color.White;
308 lblAQValue.Location = new Point(23, 24);
309 lblAQValue.Name = "lblAQValue";
310 lblAQValue.Size = new Size(617, 95);
311 lblAQValue.TabIndex = 1;
312 lblAQValue.Text = "-";
313 lblAQValue.TextAlign = ContentAlignment.TopCenter;
314 //
315 // lblPM25Value
316 //
317 lblPM25Value.Font = new Font("Segoe UI", 11F, FontStyle.Italic);
318 lblPM25Value.ForeColor = Color.FromArgb(200, 220, 240);
319 lblPM25Value.Location = new Point(23, 16);
320 lblPM25Value.Name = "lblPM25Value";
321 lblPM25Value.Size = new Size(617, 33);
322 lblPM25Value.TabIndex = 0;
323 lblPM25Value.Text = "PM 2.5: -";
324 lblPM25Value.TextAlign = ContentAlignment.MiddleLeft;
325 //
326 // lblPM10Value
327 //
328 lblPM10Value.Font = new Font("Segoe UI", 11F, FontStyle.Italic);
329 lblPM10Value.ForeColor = Color.FromArgb(200, 220, 240);
330 lblPM10Value.Location = new Point(23, 49);
331 lblPM10Value.Name = "lblPM10Value";
332 lblPM10Value.Size = new Size(617, 33);
333 lblPM10Value.TabIndex = 1;
334 lblPM10Value.Text = "PM10: -";
335 lblPM10Value.TextAlign = ContentAlignment.MiddleLeft;
336 //
337 // lblCOValue
338 //
339 lblCOValue.Font = new Font("Segoe UI", 11F, FontStyle.Italic);
340 lblCOValue.ForeColor = Color.FromArgb(200, 220, 240);
341 lblCOValue.Location = new Point(23, 82);
342 lblCOValue.Name = "lblCOValue";
343 lblCOValue.Size = new Size(617, 33);
344 lblCOValue.TabIndex = 2;
345 lblCOValue.Text = "CO: -";
346 lblCOValue.TextAlign = ContentAlignment.MiddleLeft;
347 //
348 // lblO3Value
349 //
350 lblO3Value.Font = new Font("Segoe UI", 11F, FontStyle.Italic);
351 lblO3Value.ForeColor = Color.FromArgb(200, 220, 240);
352 lblO3Value.Location = new Point(23, 115);
353 lblO3Value.Name = "lblO3Value";
354 lblO3Value.Size = new Size(617, 33);
355 lblO3Value.TabIndex = 3;
356 lblO3Value.Text = "O3: -";
357 lblO3Value.TextAlign = ContentAlignment.MiddleLeft;
358 //
359 // lblStatus
360 //
361 lblStatus.AutoSize = true;
362 lblStatus.Font = new Font("Segoe UI", 9F);
363 lblStatus.ForeColor = Color.White;
364 lblStatus.Location = new Point(20, 1750);
365 lblStatus.Name = "lblStatus";
366 lblStatus.Size = new Size(226, 20);
367 lblStatus.TabIndex = 18;
368 lblStatus.Text = "Enter city name and press Search";

```



```

369 //
370 // panelDay1
371 //
372 panelDay1.BackColor = Color.Transparent;
373 panelDay1.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
374 panelDay1.BackgroundImageLayout = ImageLayout.Stretch;
375 panelDay1.Controls.Add(lblDay1Date);
376 panelDay1.Controls.Add(lblDay1Temp);
377 panelDay1.Controls.Add(lblDay1Condition);
378 panelDay1.ForeColor = Color.FromArgb(220, 220, 220);
379 panelDay1.Location = new Point(20, 442);
380 panelDay1.Margin = new Padding(3, 4, 3, 4);
381 panelDay1.Name = "panelDay1";
382 panelDay1.Size = new Size(217, 213);
383 panelDay1.TabIndex = 5;
384 //
385 // lblDay1Date
386 //
387 lblDay1Date.BackColor = Color.Transparent;
388 lblDay1Date.Font = new Font("Segoe UI", 9F, FontStyle.Bold);
389 lblDay1Date.ForeColor = Color.White;
390 lblDay1Date.Location = new Point(6, 13);
391 lblDay1Date.Name = "lblDay1Date";
392 lblDay1Date.Size = new Size(208, 27);
393 lblDay1Date.TabIndex = 0;
394 lblDay1Date.Text = "N/A";
395 lblDay1Date.TextAlign = ContentAlignment.MiddleCenter;
396 //
397 // lblDay1Temp
398 //
399 lblDay1Temp.BackColor = Color.Transparent;
400 lblDay1Temp.Font = new Font("Segoe UI", 11F, FontStyle.Bold);
401 lblDay1Temp.ForeColor = Color.White;
402 lblDay1Temp.Location = new Point(6, 133);
403 lblDay1Temp.Name = "lblDay1Temp";
404 lblDay1Temp.Size = new Size(208, 33);
405 lblDay1Temp.TabIndex = 1;
406 lblDay1Temp.Text = "N/A";
407 lblDay1Temp.TextAlign = ContentAlignment.MiddleCenter;
408 //
409 // lblDay1Condition
410 //
411 lblDay1Condition.BackColor = Color.Transparent;
412 lblDay1Condition.Font = new Font("Segoe UI", 8F);
413 lblDay1Condition.ForeColor = Color.FromArgb(220, 230, 240);
414 lblDay1Condition.Location = new Point(6, 173);
415 lblDay1Condition.Name = "lblDay1Condition";
416 lblDay1Condition.Size = new Size(208, 33);
417 lblDay1Condition.TabIndex = 2;
418 lblDay1Condition.Text = "N/A";
419 lblDay1Condition.TextAlign = ContentAlignment.TopCenter;
420 //
421 // panelDay2
422 //
423 panelDay2.BackColor = Color.Transparent;
424 panelDay2.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
425 panelDay2.BackgroundImageLayout = ImageLayout.Stretch;
426 panelDay2.Controls.Add(lblDay2Date);
427 panelDay2.Controls.Add(lblDay2Temp);
428 panelDay2.Controls.Add(lblDay2Condition);
429 panelDay2.ForeColor = Color.FromArgb(220, 220, 220);
430 panelDay2.Location = new Point(243, 442);
431 panelDay2.Margin = new Padding(3, 4, 3, 4);
432 panelDay2.Name = "panelDay2";
433 panelDay2.Size = new Size(217, 213);
434 panelDay2.TabIndex = 6;
435 //
436 // lblDay2Date
437 //
438 lblDay2Date.BackColor = Color.Transparent;
439 lblDay2Date.Font = new Font("Segoe UI", 9F, FontStyle.Bold);
440 lblDay2Date.ForeColor = Color.White;
441 lblDay2Date.Location = new Point(6, 13);
442 lblDay2Date.Name = "lblDay2Date";
443 lblDay2Date.Size = new Size(208, 27);

```

```

444         lblDay2Date.TabIndex = 0;
445         lblDay2Date.Text = "N/A";
446         lblDay2Date.TextAlign = ContentAlignment.MiddleCenter;
447         //
448         // lblDay2Temp
449         //
450         lblDay2Temp.BackColor = Color.Transparent;
451         lblDay2Temp.Font = new Font("Segoe UI", 11F, FontStyle.Bold);
452         lblDay2Temp.ForeColor = Color.White;
453         lblDay2Temp.Location = new Point(6, 133);
454         lblDay2Temp.Name = "lblDay2Temp";
455         lblDay2Temp.Size = new Size(208, 33);
456         lblDay2Temp.TabIndex = 1;
457         lblDay2Temp.Text = "N/A";
458         lblDay2Temp.TextAlign = ContentAlignment.MiddleCenter;
459         //
460         // lblDay2Condition
461         //
462         lblDay2Condition.BackColor = Color.Transparent;
463         lblDay2Condition.Font = new Font("Segoe UI", 8F);
464         lblDay2Condition.ForeColor = Color.FromArgb(220, 230, 240);
465         lblDay2Condition.Location = new Point(6, 173);
466         lblDay2Condition.Name = "lblDay2Condition";
467         lblDay2Condition.Size = new Size(208, 33);
468         lblDay2Condition.TabIndex = 2;
469         lblDay2Condition.Text = "N/A";
470         lblDay2Condition.TextAlign = ContentAlignment.TopCenter;
471         //
472         // panelDay3
473         //
474         panelDay3.BackColor = Color.Transparent;
475         panelDay3.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design_3_;
476         panelDay3.BackgroundImageLayout = ImageLayout.Stretch;
477         panelDay3.Controls.Add(lblDay3Date);
478         panelDay3.Controls.Add(lblDay3Temp);
479         panelDay3.Controls.Add(lblDay3Condition);
480         panelDay3.ForeColor = Color.FromArgb(220, 220, 220);
481         panelDay3.Location = new Point(466, 442);
482         panelDay3.Margin = new Padding(3, 4, 3, 4);
483         panelDay3.Name = "panelDay3";
484         panelDay3.Size = new Size(217, 213);
485         panelDay3.TabIndex = 7;
486         //
487         // lblDay3Date
488         //
489         lblDay3Date.Font = new Font("Segoe UI", 9F, FontStyle.Bold);
490         lblDay3Date.ForeColor = Color.White;
491         lblDay3Date.Location = new Point(6, 13);
492         lblDay3Date.Name = "lblDay3Date";
493         lblDay3Date.Size = new Size(208, 27);
494         lblDay3Date.TabIndex = 0;
495         lblDay3Date.Text = "N/A";
496         lblDay3Date.TextAlign = ContentAlignment.MiddleCenter;
497         //
498         // lblDay3Temp
499         //
500         lblDay3Temp.Font = new Font("Segoe UI", 11F, FontStyle.Bold);
501         lblDay3Temp.ForeColor = Color.White;
502         lblDay3Temp.Location = new Point(6, 133);
503         lblDay3Temp.Name = "lblDay3Temp";
504         lblDay3Temp.Size = new Size(208, 33);
505         lblDay3Temp.TabIndex = 1;
506         lblDay3Temp.Text = "N/A";
507         lblDay3Temp.TextAlign = ContentAlignment.MiddleCenter;
508         //
509         // lblDay3Condition
510         //
511         lblDay3Condition.Font = new Font("Segoe UI", 8F);
512         lblDay3Condition.ForeColor = Color.FromArgb(220, 230, 240);
513         lblDay3Condition.Location = new Point(6, 173);
514         lblDay3Condition.Name = "lblDay3Condition";
515         lblDay3Condition.Size = new Size(208, 33);
516         lblDay3Condition.TabIndex = 2;
517         lblDay3Condition.Text = "N/A";

```

```

518         lblDay3Condition.TextAlign = ContentAlignment.TopCenter;
519         //
520         // panelForecast
521         //
522         panelForecast.BackColor = Color.Transparent;
523         panelForecast.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
524         panelForecast.BackgroundImageLayout = ImageLayout.Stretch;
525         panelForecast.Controls.Add(lblLastUpdate);
526         panelForecast.Controls.Add(lblCity);
527         panelForecast.Controls.Add(lblCountry);
528         panelForecast.Controls.Add(lblTemp);
529         panelForecast.Controls.Add(lblCondition);
530         panelForecast.Controls.Add(lblFeelsLike);
531         panelForecast.ForeColor = Color.White;
532         panelForecast.Location = new Point(20, 123);
533         panelForecast.Margin = new Padding(3, 4, 3, 4);
534         panelForecast.Name = "panelForecast";
535         panelForecast.Size = new Size(663, 293);
536         panelForecast.TabIndex = 4;
537         //
538         // lblLastUpdate
539         //
540         lblLastUpdate.AutoSize = true;
541         lblLastUpdate.Font = new Font("Segoe UI", 10.2F, FontStyle.Regular, GraphicsUnit.Point, 0);
542         lblLastUpdate.ForeColor = Color.FromArgb(220, 230, 240);
543         lblLastUpdate.Location = new Point(447, 229);
544         lblLastUpdate.Name = "lblLastUpdate";
545         lblLastUpdate.Size = new Size(17, 23);
546         lblLastUpdate.TabIndex = 6;
547         lblLastUpdate.Text = "-";
548         //
549         // panelUV
550         //
551         panelUV.BackColor = Color.Transparent;
552         panelUV.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
553         panelUV.BackgroundImageLayout = ImageLayout.Stretch;
554         panelUV.Controls.Add(picUV);
555         panelUV.Controls.Add(lblTitleUV);
556         panelUV.Controls.Add(lblUVValue);
557         panelUV.ForeColor = Color.FromArgb(220, 220, 220);
558         panelUV.Location = new Point(20, 688);
559         panelUV.Margin = new Padding(3, 4, 3, 4);
560         panelUV.Name = "panelUV";
561         panelUV.Size = new Size(320, 213);
562         panelUV.TabIndex = 10;
563         //
564         // picUV
565         //
566         picUV.BackgroundImageLayout = ImageLayout.Zoom;
567         picUV.ErrorImage = null;
568         picUV.InitialImage = null;
569         picUV.Location = new Point(144, 3);
570         picUV.Name = "picUV";
571         picUV.Size = new Size(173, 207);
572         picUV.SizeMode = PictureBoxSizeMode.Zoom;
573         picUV.TabIndex = 8;
574         picUV.TabStop = false;
575         //
576         // lblTitleUV
577         //
578         lblTitleUV.Font = new Font("Segoe UI", 18F, FontStyle.Bold, GraphicsUnit.Point, 0);
579         lblTitleUV.ForeColor = Color.Transparent;
580         lblTitleUV.Location = new Point(0, 0);
581         lblTitleUV.Name = "lblTitleUV";
582         lblTitleUV.Size = new Size(61, 55);
583         lblTitleUV.TabIndex = 2;
584         lblTitleUV.Text = "UV";
585         lblTitleUV.TextAlign = ContentAlignment.BottomLeft;
586         //
587         // panelHumidity
588         //
589         panelHumidity.BackColor = Color.Transparent;
590         panelHumidity.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design__3_;
591         panelHumidity.BackgroundImageLayout = ImageLayout.Stretch;
592         panelHumidity.Controls.Add(picHumidity);

```

```

593     panelHumidity.Controls.Add(lblTitleHumidity);
594     panelHumidity.Controls.Add(lblHumidityValue);
595     panelHumidity.ForeColor = Color.FromArgb(220, 220, 220);
596     panelHumidity.Location = new Point(363, 688);
597     panelHumidity.Margin = new Padding(3, 4, 3, 4);
598     panelHumidity.Name = "panelHumidity";
599     panelHumidity.Size = new Size(320, 213);
600     panelHumidity.TabIndex = 11;
601     //
602     // picHumidity
603     //
604     picHumidity.BackColor = Color.Transparent;
605     picHumidity.BackgroundImageLayout = ImageLayout.Zoom;
606     picHumidity.ErrorImage = null;
607     picHumidity.InitialImage = null;
608     picHumidity.Location = new Point(148, 3);
609     picHumidity.Name = "picHumidity";
610     picHumidity.Size = new Size(172, 207);
611     picHumidity.SizeMode = PictureBoxSizeMode.Zoom;
612     picHumidity.TabIndex = 9;
613     picHumidity.TabStop = false;
614     //
615     // lblTitleHumidity
616     //
617     lblTitleHumidity.Font = new Font("Segoe UI", 18F, FontStyle.Bold, GraphicsUnit.Point, 0);
618     lblTitleHumidity.ForeColor = Color.Transparent;
619     lblTitleHumidity.Location = new Point(0, 0);
620     lblTitleHumidity.Name = "lblTitleHumidity";
621     lblTitleHumidity.Size = new Size(155, 55);
622     lblTitleHumidity.TabIndex = 3;
623     lblTitleHumidity.Text = "Humidity";
624     lblTitleHumidity.TextAlign = ContentAlignment.BottomLeft;
625     //
626     // panelPressure
627     //
628     panelPressure.BackColor = Color.Transparent;
629     panelPressure.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design_3_;
630     panelPressure.BackgroundImageLayout = ImageLayout.Stretch;
631     panelPressure.Controls.Add(picPressure);
632     panelPressure.Controls.Add(lblTitlePressure);
633     panelPressure.Controls.Add(lblPressureValue);
634     panelPressure.ForeColor = Color.FromArgb(220, 220, 220);
635     panelPressure.Location = new Point(20, 928);
636     panelPressure.Margin = new Padding(3, 4, 3, 4);
637     panelPressure.Name = "panelPressure";
638     panelPressure.Size = new Size(320, 213);
639     panelPressure.TabIndex = 12;
640     //
641     // picPressure
642     //
643     picPressure.BackgroundImageLayout = ImageLayout.Stretch;
644     picPressure.ErrorImage = null;
645     picPressure.InitialImage = null;
646     picPressure.Location = new Point(144, 3);
647     picPressure.Name = "picPressure";
648     picPressure.Size = new Size(173, 207);
649     picPressure.SizeMode = PictureBoxSizeMode.Zoom;
650     picPressure.TabIndex = 9;
651     picPressure.TabStop = false;
652     //
653     // lblTitlePressure
654     //
655     lblTitlePressure.Font = new Font("Segoe UI", 18F, FontStyle.Bold, GraphicsUnit.Point, 0);
656     lblTitlePressure.ForeColor = Color.Transparent;
657     lblTitlePressure.Location = new Point(0, 0);
658     lblTitlePressure.Name = "lblTitlePressure";
659     lblTitlePressure.Size = new Size(155, 55);
660     lblTitlePressure.TabIndex = 4;
661     lblTitlePressure.Text = "Pressure";
662     lblTitlePressure.TextAlign = ContentAlignment.BottomLeft;
663     //
664     // panelCloud
665     //
666     panelCloud.BackColor = Color.Transparent;

```

```

667 panelCloud.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design_3_;
668 panelCloud.BackgroundImageLayout = ImageLayout.Stretch;
669 panelCloud.Controls.Add(picCloud);
670 panelCloud.Controls.Add(lblTitleCloud);
671 panelCloud.Controls.Add(lblCloudValue);
672 panelCloud.ForeColor = Color.FromArgb(220, 220, 220);
673 panelCloud.Location = new Point(363, 928);
674 panelCloud.Margin = new Padding(3, 4, 3, 4);
675 panelCloud.Name = "panelCloud";
676 panelCloud.Size = new Size(320, 213);
677 panelCloud.TabIndex = 13;
678 //
679 // picCloud
680 //
681 picCloud.BackgroundImageLayout = ImageLayout.Zoom;
682 picCloud.ErrorImage = null;
683 picCloud.InitialImage = null;
684 picCloud.Location = new Point(148, 3);
685 picCloud.Name = "picCloud";
686 picCloud.Size = new Size(169, 207);
687 picCloud.SizeMode = PictureBoxSizeMode.Zoom;
688 picCloud.TabIndex = 9;
689 picCloud.TabStop = false;
690 //
691 // lblTitleCloud
692 //
693 lblTitleCloud.Font = new Font("Segoe UI", 18F, FontStyle.Bold, GraphicsUnit.Point, 0);
694 lblTitleCloud.ForeColor = Color.Transparent;
695 lblTitleCloud.Location = new Point(0, 0);
696 lblTitleCloud.Name = "lblTitleCloud";
697 lblTitleCloud.Size = new Size(155, 55);
698 lblTitleCloud.TabIndex = 4;
699 lblTitleCloud.Text = "Cloud";
700 lblTitleCloud.TextAlign = ContentAlignment.BottomLeft;
701 //
702 // panelWind
703 //
704 panelWind.BackColor = Color.Transparent;
705 panelWind.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design_3_;
706 panelWind.BackgroundImageLayout = ImageLayout.Stretch;
707 panelWind.Controls.Add(picWind);
708 panelWind.Controls.Add(lblTitleWind);
709 panelWind.Controls.Add(lblWindValue);
710 panelWind.ForeColor = Color.FromArgb(220, 220, 220);
711 panelWind.Location = new Point(20, 1168);
712 panelWind.Margin = new Padding(3, 4, 3, 4);
713 panelWind.Name = "panelWind";
714 panelWind.Size = new Size(320, 213);
715 panelWind.TabIndex = 14;
716 //
717 // picWind
718 //
719 picWind.BackgroundImageLayout = ImageLayout.Zoom;
720 picWind.ErrorImage = null;
721 picWind.InitialImage = null;
722 picWind.Location = new Point(161, 3);
723 picWind.Name = "picWind";
724 picWind.Size = new Size(156, 207);
725 picWind.SizeMode = PictureBoxSizeMode.Zoom;
726 picWind.TabIndex = 9;
727 picWind.TabStop = false;
728 //
729 // lblTitleWind
730 //
731 lblTitleWind.Font = new Font("Segoe UI", 18F, FontStyle.Bold, GraphicsUnit.Point, 0);
732 lblTitleWind.ForeColor = Color.Transparent;
733 lblTitleWind.Location = new Point(0, 0);
734 lblTitleWind.Name = "lblTitleWind";
735 lblTitleWind.Size = new Size(155, 55);
736 lblTitleWind.TabIndex = 4;
737 lblTitleWind.Text = "Wind";
738 lblTitleWind.TextAlign = ContentAlignment.BottomLeft;
739 //
740 // panelPrecip
741 //

```

```

742     panelPrecip.BackColor = Color.Transparent;
743     panelPrecip.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design_3_;
744     panelPrecip.BackgroundImageLayout = ImageLayout.Stretch;
745     panelPrecip.Controls.Add(picPrecip);
746     panelPrecip.Controls.Add(lblTitlePrecip);
747     panelPrecip.Controls.Add(lblPrecipValue);
748     panelPrecip.ForeColor = Color.FromArgb(220, 220, 220);
749     panelPrecip.Location = new Point(363, 1168);
750     panelPrecip.Margin = new Padding(3, 4, 3, 4);
751     panelPrecip.Name = "panelPrecip";
752     panelPrecip.Size = new Size(320, 213);
753     panelPrecip.TabIndex = 15;
754     //
755     // picPrecip
756     //
757     picPrecip.BackgroundImageLayout = ImageLayout.Zoom;
758     picPrecip.ErrorImage = null;
759     picPrecip.InitialImage = null;
760     picPrecip.Location = new Point(148, 55);
761     picPrecip.Name = "picPrecip";
762     picPrecip.Size = new Size(172, 155);
763     picPrecip.SizeMode = PictureBoxSizeMode.Zoom;
764     picPrecip.TabIndex = 9;
765     picPrecip.TabStop = false;
766     //
767     // lblTitlePrecip
768     //
769     lblTitlePrecip.Font = new Font("Segoe UI", 18F, FontStyle.Bold, GraphicsUnit.Point, 0);
770     lblTitlePrecip.ForeColor = Color.Transparent;
771     lblTitlePrecip.Location = new Point(0, 0);
772     lblTitlePrecip.Name = "lblTitlePrecip";
773     lblTitlePrecip.Size = new Size(204, 55);
774     lblTitlePrecip.TabIndex = 4;
775     lblTitlePrecip.Text = "Precipitation ";
776     lblTitlePrecip.TextAlign = ContentAlignment.BottomLeft;
777     //
778     // panelAirQuality
779     //
780     panelAirQuality.BackColor = Color.Transparent;
781     panelAirQuality.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design_3_;
782     panelAirQuality.BackgroundImageLayout = ImageLayout.Stretch;
783     panelAirQuality.Controls.Add(lblAQValue);
784     panelAirQuality.ForeColor = Color.FromArgb(220, 220, 220);
785     panelAirQuality.Location = new Point(20, 1404);
786     panelAirQuality.Margin = new Padding(3, 4, 3, 4);
787     panelAirQuality.Name = "panelAirQuality";
788     panelAirQuality.Size = new Size(663, 133);
789     panelAirQuality.TabIndex = 16;
790     //
791     // panelAirDetails
792     //
793     panelAirDetails.BackColor = Color.Transparent;
794     panelAirDetails.BackgroundImage = ProjectAkhir.Properties.Resources.Untitled_design_3_;
795     panelAirDetails.BackgroundImageLayout = ImageLayout.Stretch;
796     panelAirDetails.Controls.Add(lblPM25Value);
797     panelAirDetails.Controls.Add(lblPM10Value);
798     panelAirDetails.Controls.Add(lblCOValue);
799     panelAirDetails.Controls.Add(lblO3Value);
800     panelAirDetails.ForeColor = Color.FromArgb(220, 220, 220);
801     panelAirDetails.Location = new Point(20, 1557);
802     panelAirDetails.Margin = new Padding(3, 4, 3, 4);
803     panelAirDetails.Name = "panelAirDetails";
804     panelAirDetails.Size = new Size(663, 173);
805     panelAirDetails.TabIndex = 17;
806     //
807     // Form1
808     //
809     AutoScaleDimensions = new.SizeF(8F, 20F);
810     AutoScaleMode = AutoScaleMode.Font;
811     AutoScroll = true;
812     BackColor = Color.FromArgb(52, 108, 160);
813     BackgroundImage = ProjectAkhir.Resources.Resources.bg;
814     BackgroundImageLayout = ImageLayout.Stretch;
815     ClientSize = new Size(718, 1055);
816     Controls.Add(lblTitlePrecip);

```



```

816 Controls.Add(lblStatus);
817 Controls.Add(panelAirDetails);
818 Controls.Add(panelAirQuality);
819 Controls.Add(panelPrecip);
820 Controls.Add(panelWind);
821 Controls.Add(panelCloud);
822 Controls.Add(panelPressure);
823 Controls.Add(panelHumidity);
824 Controls.Add(panelUV);
825 Controls.Add(panelDay3);
826 Controls.Add(panelDay2);
827 Controls.Add(panelDay1);
828 Controls.Add(panelForecast);
829 Controls.Add(btnSearch);
830 Controls.Add(txtCity);
831 Controls.Add(lblTitle);
832 Icon = (Icon)resources.GetObject("$this.Icon");
833 Margin = new Padding(3, 4, 3, 4);
834 Name = "Form1";
835 StartPosition = FormStartPosition.CenterScreen;
836 Text = "Weather";
837 panelDay1.ResumeLayout(false);
838 panelDay2.ResumeLayout(false);
839 panelDay3.ResumeLayout(false);
840 panelForecast.ResumeLayout(false);
841 panelForecast.PerformLayout();
842 panelUV.ResumeLayout(false);
843 ((System.ComponentModel.ISupportInitialize)picUV).EndInit();
844 panelHumidity.ResumeLayout(false);
845 ((System.ComponentModel.ISupportInitialize)picHumidity).EndInit();
846 panelPressure.ResumeLayout(false);
847 ((System.ComponentModel.ISupportInitialize)picPressure).EndInit();
848 panelCloud.ResumeLayout(false);
849 ((System.ComponentModel.ISupportInitialize)picCloud).EndInit();
850 panelWind.ResumeLayout(false);
851 ((System.ComponentModel.ISupportInitialize)picWind).EndInit();
852 panelPrecip.ResumeLayout(false);
853 ((System.ComponentModel.ISupportInitialize)picPrecip).EndInit();
854 panelAirQuality.ResumeLayout(false);
855 panelAirDetails.ResumeLayout(false);
856 ResumeLayout(false);
857 PerformLayout();
858 }
859
860 private System.Windows.Forms.Label CreateCardLabel(string text, int top, int width)
861 {
862     var label = new System.Windows.Forms.Label();
863     label.Font = new System.Drawing.Font("Segoe UI", 13F, System.Drawing.FontStyle.Bold);
864     label.ForeColor = System.Drawing.Color.FromArgb(100, 130, 160);
865     label.Location = new System.Drawing.Point((280 - width) / 2, top);
866     label.Name = "lbl" + text + "Title";
867     label.Size = new System.Drawing.Size(width, 25);
868     label.TabIndex = 0;
869     label.Text = text.ToUpper();
870     label.TextAlign = System.Drawing.ContentAlignment.MiddleCenter;
871     return label;
872 }
873 private Label lblTitleUV;
874 private Label lblTitleHumidity;
875 private Label lblTitlePressure;
876 private Label lblTitleCloud;
877 private Label lblTitleWind;
878 private Label lblTitlePrecip;
879 private Label lblLastUpdate;
880 private PictureBox picUV;
881 private PictureBox picHumidity;
882 private PictureBox picPressure;
883 private PictureBox picCloud;
884 private PictureBox picWind;
885 private PictureBox picPrecip;
886 }
887 }

```

4. WeatherIcon.cs

```
1  using System;
2  using System.Collections.Generic;
3  using System.Drawing;
4  using System.IO;
5  using System.Windows.Forms;
6  using WeatherApp;
7
8  namespace WeatherApp
9  {
10     public static class WeatherIcon
11     {
12         // Dictionary untuk menyimpan mapping antara kode cuaca dengan nama icon
13         private static Dictionary<int, string> weatherCodeToIcon = new Dictionary<int, string>();
14         // Variable untuk menyimpan path folder icon
15         private static string iconFolder = Path.Combine(Application.StartupPath, "icons");
16
17         // Constructor static yang dipanggil pertama kali saat class digunakan
18         static WeatherIcon()
19         {
20             InitializeWeatherCodes();
21         }
22
23         // Fungsi untuk mengatur path folder icon
24         public static void IconFolder(string folderPath)
25         {
26             iconFolder = folderPath;
27         }
28
29         // Fungsi untuk menginisialisasi mapping kode cuaca ke nama icon
30         private static void InitializeWeatherCodes()
31         {
32             weatherCodeToIcon = new Dictionary<int, string>
33             {
34                 // Cerah/Berawan
35                 { 1000, "clear-day" },
36
37                 // Cerah Berawan
38                 { 1003, "partly-cloudy-day" },
39
40                 // Berawan
41                 { 1006, "cloudy" },
42                 { 1009, "overcast" },
43
44                 // Kabut
45                 { 1030, "mist" },
46                 { 1135, "fog" },
47                 { 1147, "fog" },
48
49                 // Gerimis
50                 { 1063, "drizzle" },
51                 { 1150, "drizzle" },
52                 { 1153, "drizzle" },
53                 { 1168, "sleet" },
54                 { 1171, "sleet" },
55
56                 // Hujan
57                 { 1180, "rain" },
58                 { 1183, "rain" },
59                 { 1186, "rain" },
60                 { 1189, "rain" },
61                 { 1192, "rain" },
62                 { 1195, "rain" },
63                 { 1198, "sleet" },
64                 { 1201, "sleet" },
65                 { 1240, "rain" },
66                 { 1243, "rain" },
67                 { 1246, "rain" },
68
69                 // Salju
70                 { 1066, "snow" },
71                 { 1069, "sleet" },
72                 { 1072, "sleet" },
73                 { 1114, "snow" },
74                 { 1117, "snow" },
75                 { 1204, "sleet" },
76                 { 1207, "sleet" },
77                 { 1210, "snow" },
78                 { 1213, "snow" },
```



```

79         { 1216, "snow" },
80         { 1219, "snow" },
81         { 1222, "snow" },
82         { 1225, "snow" },
83         { 1237, "hail" },
84         { 1249, "sleet" },
85         { 1252, "sleet" },
86         { 1255, "snow" },
87         { 1258, "snow" },
88         { 1261, "hail" },
89         { 1264, "hail" },
90
91         // Badai Petir
92         { 1087, "thunderstorms-day" },
93         { 1273, "thunderstorms-day-rain" },
94         { 1276, "thunderstorms-rain" },
95         { 1279, "thunderstorms-day-snow" },
96         { 1282, "thunderstorms-snow" }
97     };
98 }
99
100 // Fungsi untuk memuat icon cuaca berdasarkan kode cuaca dan waktu (siang/malam)
101 public static Image LoadWeatherIcon(int weatherCode, bool isDay)
102 {
103     try
104     {
105         // Dapatkan nama icon berdasarkan kode cuaca dan waktu
106         string iconName = GetIconName(weatherCode, isDay);
107         // Gabungkan path folder dengan nama file icon
108         string iconPath = Path.Combine(iconFolder, $"{iconName}.png");
109
110         // Cek apakah file icon ada
111         if (File.Exists(iconPath))
112         {
113             // Buka file icon dan buat copy bitmap-nya
114             using (var originalImage = Image.FromFile(iconPath))
115             {
116                 return new Bitmap(originalImage);
117             }
118         }
119         else
120         {
121             // Tulis pesan debug jika icon tidak ditemukan
122             System.Diagnostics.Debug.WriteLine($"Icon not found: {iconPath}");
123             return null;
124         }
125     }
126     catch (Exception ex)
127     {
128         // Tangkap error dan tulis pesan debug
129         System.Diagnostics.Debug.WriteLine($"Error loading icon: {ex.Message}");
130         return null;
131     }
132 }
133
134 // Fungsi untuk memuat icon cuaca dengan ukuran custom (resize)
135 public static Image LoadWeatherIcon(int weatherCode, bool isDay, int width, int height)
136 {
137     try
138     {
139         // Muat icon asli terlebih dahulu
140         Image originalIcon = LoadWeatherIcon(weatherCode, isDay);
141         if (originalIcon == null) return null;
142
143         // Buat bitmap baru dengan ukuran yang diinginkan
144         Bitmap resizedIcon = new Bitmap(width, height);
145         using (Graphics g = Graphics.FromImage(resizedIcon))
146         {
147             // Gunakan interpolasi berkualitas tinggi untuk hasil resize yang bagus
148             g.InterpolationMode = System.Drawing.Drawing2D.InterpolationMode.HighQualityBicubic;
149             // Gambar icon asli ke bitmap baru dengan ukuran baru
150             g.DrawImage(originalIcon, 0, 0, width, height);
151         }
152
153         // Hapus icon asli dari memory untuk menghemat resource
154         originalIcon.Dispose();
155         return resizedIcon;
156     }
157     catch (Exception ex)
158     {
159         // Tangkap error dan tulis pesan debug
160         System.Diagnostics.Debug.WriteLine($"Error resizing icon: {ex.Message}");

```

```

161         return null!;
162     }
163 }
164
165 // Fungsi private untuk mendapatkan nama icon berdasarkan kode cuaca dan waktu
166 private static string GetIconName(int weatherCode, bool isDay)
167 {
168     string iconName;
169
170     // Cari nama icon dari dictionary berdasarkan kode cuaca
171     if (!weatherCodeToIcon.TryGetValue(weatherCode, out iconName!))
172     {
173         // Jika kode cuaca tidak ditemukan, gunakan icon default
174         System.Diagnostics.Debug.WriteLine($"⚠ Weather code {weatherCode} NOT FOUND in mapping! Using default.");
175         iconName = isDay ? "clear-day" : "clear-night";
176     }
177     else
178     {
179         // Tulis pesan debug jika mapping berhasil ditemukan
180         System.Diagnostics.Debug.WriteLine($"✓ Weather code {weatherCode} mapped to: {iconName}");
181     }
182
183     // Ganti icon siang dengan icon malam jika waktu adalah malam
184     if (!isDay)
185     {
186         if (weatherCode == 1000) // Cerah
187         {
188             iconName = "clear-night";
189         }
190         else if (weatherCode == 1003) // Cerah Berawan
191         {
192             iconName = "partly-cloudy-night";
193         }
194         else if (weatherCode == 1087) // Badai Petir
195         {
196             iconName = "thunderstorms-night";
197         }
198         else if (weatherCode == 1273) // Badai Petir dengan Hujan
199         {
200             iconName = "thunderstorms-night-rain";
201         }
202         else if (weatherCode == 1279) // Badai Petir dengan Salju
203         {
204             iconName = "thunderstorms-night-snow";
205         }
206         else if (iconName.Contains("-day"))
207         {
208             // Ganti semua icon yang mengandung "-day" menjadi "-night"
209             iconName = iconName.Replace("-day", "-night");
210         }
211     }
212
213     return iconName;
214 }
215
216 // Fungsi untuk mendapatkan daftar icon yang diperlukan untuk didownload
217 public static List<string> GetRequiredIcons()
218 {
219     return new List<string>
220     {
221         // Icon untuk siang hari
222         "clear-day.png",
223         "partly-cloudy-day.png",
224         "cloudy.png",
225         "overcast.png",
226         "mist.png",
227         "fog.png",
228         "drizzle.png",
229         "rain.png",
230         "snow.png",
231         "sleet.png",
232         "hail.png",
233         "thunderstorms-day.png",
234         "thunderstorms-day-rain.png",
235         "thunderstorms-day-snow.png",
236         "thunderstorms-rain.png",
237         "thunderstorms-snow.png",
238
239         // Icon untuk malam hari
240         "clear-night.png",
241         "partly-cloudy-night.png",
242         "thunderstorms-night.png",
243         "thunderstorms-night-rain.png"
244     }
245 }

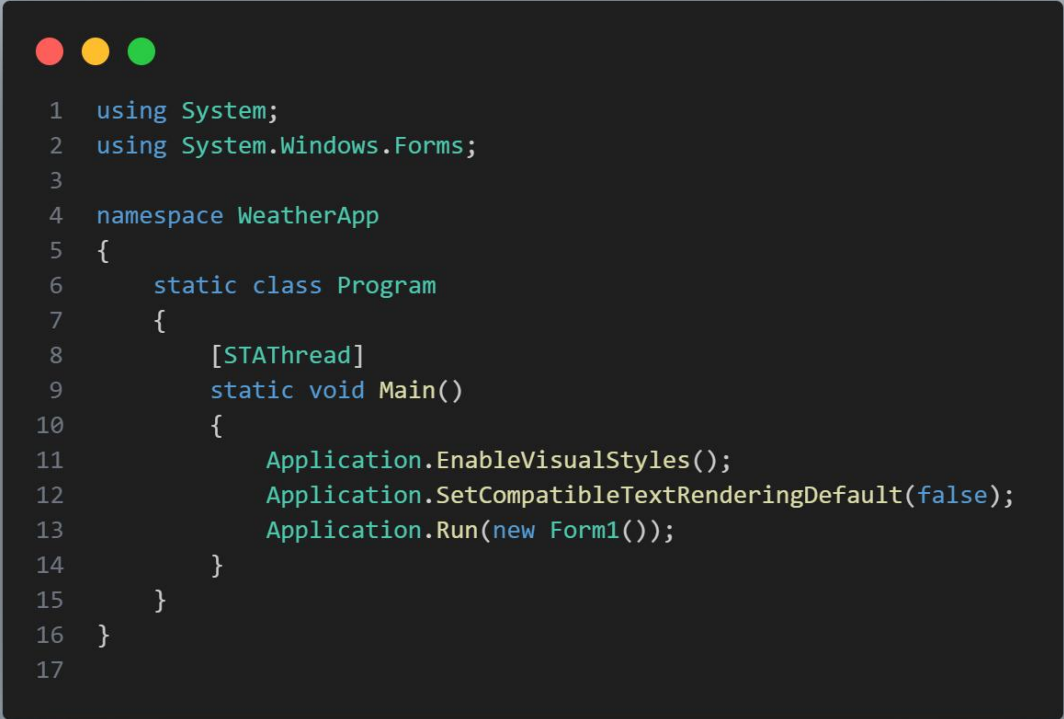
```

```

243         "thunderstorms-night-rain.png";
244         "thunderstorms-night-snow.png"
245     };
246 }
247
248 // Fungsi untuk mengecek apakah icon tersedia di folder
249 public static bool IsIconAvailable(int weatherCode, bool isDay)
250 {
251     // Dapatkan nama icon
252     string iconName = GetIconName(weatherCode, isDay);
253     // Gabungkan path folder dengan nama file
254     string iconPath = Path.Combine(iconFolder, $"{iconName}.png");
255     // Return true jika file ada, false jika tidak ada
256     return File.Exists(iconPath);
257 }
258
259 // Fungsi untuk mendapatkan path icon berdasarkan deskripsi kondisi cuaca
260 public static string GetIconPath(string condition)
261 {
262     // Ubah kondisi menjadi huruf kecil untuk pencocokan yang lebih mudah
263     condition = condition.ToLower();
264     string iconName = "not-available.png"; // icon default
265
266     // Cek kondisi cuaca dan tentukan icon yang sesuai
267     if (condition.Contains("sunny") || condition.Contains("clear"))
268         iconName = "day-clear.png";
269     else if (condition.Contains("cloud"))
270         iconName = "cloudy.png";
271     else if (condition.Contains("rain"))
272         iconName = "rain.png";
273     else if (condition.Contains("thunder"))
274         iconName = "thunderstorms.png";
275     else if (condition.Contains("snow"))
276         iconName = "snow.png";
277     else if (condition.Contains("mist") || condition.Contains("fog"))
278         iconName = "fog.png";
279
280     // Gabungkan path folder dengan nama icon
281     return Path.Combine(Application.StartupPath, "Icons");
282 }
283
284 // Deskripsi cuaca dalam Bahasa Inggris
285 public static string GetWeatherDescription(int weatherCode)
286 {
287     // Dictionary untuk menyimpan deskripsi cuaca dalam Bahasa Inggris
288     var descriptions = new Dictionary<int, string>
289     {
290         { 1000, "Clear" },
291         { 1003, "Partly Cloudy" },
292         { 1006, "Cloudy" },
293         { 1009, "Overcast" },
294         { 1030, "Mist" },
295         { 1063, "Possible Rain" },
296         { 1135, "Fog" },
297         { 1147, "Freezing Fog" },
298         { 1150, "Light Drizzle" },
299         { 1153, "Light Drizzle" },
300         { 1180, "Light Rain" },
301         { 1183, "Light Rain" },
302         { 1186, "Moderate Rain" },
303         { 1189, "Moderate Rain" },
304         { 1192, "Heavy Rain" },
305         { 1195, "Heavy Rain" },
306         { 1087, "Thunderstorm" },
307         { 1210, "Light Snow" },
308         { 1213, "Light Snow" },
309         { 1219, "Moderate Snow" },
310         { 1225, "Heavy Snow" },
311         { 1237, "Hail" },
312         { 1273, "Thunder Rain" },
313         { 1276, "Heavy Thunder Rain" }
314     };
315
316     // Return deskripsi jika kode cuaca ditemukan, jika tidak return "Unknown"
317     return descriptions.ContainsKey(weatherCode)
318         ? descriptions[weatherCode]
319         : "Unknown";
320 }
321 }
322 }

```

5. Program.cs



```
1  using System;
2  using System.Windows.Forms;
3
4  namespace WeatherApp
5  {
6      static class Program
7      {
8          [STAThread]
9          static void Main()
10         {
11             Application.EnableVisualStyles();
12             Application.SetCompatibleTextRenderingDefault(false);
13             Application.Run(new Form1());
14         }
15     }
16 }
17
```

Screenshot Tampilan Aplikasi

